



BitInsight

Crypto Trading & Wallet Platform

Java / Spring Boot Project Interview Handbook

A premium revision guide for fresher Java full-stack developers

PROJECT	BitInsight — Crypto Trading & Wallet Platform
AUTHOR	Balaram Gochhayat
GIT BRANCH	Balaram_Treading_Application
AUDIENCE	Junior / Fresher Java Full-Stack Developer
BACKEND	Java 21 · Spring Boot 3.4.5 · Spring Security 6 · Spring Data JPA · Hibernate · MySQL · Caffeine Cache
FRONTEND	React 19 · Vite 6 · Redux (Thunk) · Tailwind CSS · Radix UI · ApexCharts · Axios
INTEGRATIONS	CoinGecko API · Razorpay Java SDK · Stripe Java SDK · JavaMailSender

About This Handbook

This handbook turns the BitInsight codebase into a **speak-it-out-loud interview guide**. Every claim is tied to what the repository actually contains, so you can answer confidently without over-claiming.

How to Use It

If you have...	Read this
10 minutes	45. One-Page Quick Revision and 46. Top 20 Interview Questions
1 hour	Part I, Part II, then 43. Things Not to Say
A full day	Read in order, then rehearse 41. Mock Interview Simulation out loud
Right before the interview	47. Final Interview Checklist

Accuracy and Honesty Standard

Every architectural detail is tagged with its implementation status. Learn these four badges, because interviewers reward candidates who separate *what exists* from *what they would build next*.

Badge	Meaning	Where the claim comes from
 Implemented	Directly present in the repository files, configurations, or classes.	Original tag: <code>[CONFIRMED FROM CODE]</code>
 Inferred	Architectural rationale derived from actual code patterns.	Original tag: <code>[REASONABLE INFERENCE]</code>
 Partial	Feature has partial backend/frontend code or lacks production hardening.	Original tag: <code>[PARTIALLY IMPLEMENTED]</code>
 Future Improvement	Not present in the codebase. A typical interview topic or a recommended enhancement.	Original tag: <code>[NOT IMPLEMENTED / FUTURE IMPROVEMENT]</code>

Callout Legend

Key Concept

A core idea worth understanding deeply.

Interview Priority

Very likely to be asked. Prepare a spoken answer.

Common Mistake

A trap that costs candidates credibility.

Remember

A fact to keep in your head during the interview.

Interview Tip

How to phrase or position an answer for maximum impact.

Implementation Status at a Glance

Use this table as your **honesty map**. If an interviewer asks "did you build X?", check which column it belongs to.

✅ Implemented	🟡 Partial	🔍 Inferred (rationale)	🚀 Future Improvement
Spring Boot REST API (Auth, User, Coin, Order, Wallet, Payment, Withdrawal, Watchlist)	Role-based URL access: <code>/api/**</code> requires authentication, but <code>/api/admin/**</code> has no <code>hasRole("ADMIN")</code> rule	Why Java 21, Spring Boot, MySQL, Caffeine, and JWT were chosen	<code>BCryptPasswordEncoder</code> for password hashing (passwords are currently stored in plain text)
Stateless JWT authentication (HMAC-SHA, 24-hour expiry)	Password handling: login and signup work, but with plain-text storage	Behaviour under race conditions, database outage, and CoinGecko outage (scenario answers)	Enforce admin-only routes with <code>hasRole("ADMIN")</code> or <code>@PreAuthorize</code>
Email OTP two-factor authentication and OTP password reset		Performance recommendations (indexes, pagination)	Refresh tokens
Caffeine cache (<code>coinList</code> , <code>marketChart</code>) plus <code>@Scheduled</code> cache refresh			Redis distributed caching
Razorpay (INR) and Stripe (USD) deposits			WebSocket (STOMP) live price feeds
Transactional BUY/SELL orders, wallet transfers, withdrawals			JUnit 5 + Mockito tests and GitHub Actions CI/CD
<code>GlobalExceptionHandler</code> (<code>@ControllerAdvice</code>)			Docker, Docker Compose, Nginx production deployment
CORS configuration and stateless sessions			Push (Firebase FCM) and SMS notifications
React 19 SPA with Redux, Axios, ApexCharts			Withdrawal overdraft check before deducting balance

CONTENTS

Table of Contents

FRONT MATTER

- About This Handbook 1
- Implementation Status at a Glance

PART I Project Foundations

- 01 Project at a Glance
- 02 One-Minute Pitch
- 03 Detailed Walkthrough
- 04 Business Problem

05	Users and Roles
06	Technology Stack

PART II Architecture and Flow

07	System Architecture
08	End-to-End User Journey
09	Authentication Flow
10	Authorization and RBAC
11	API Request Lifecycle

PART III Implementation Deep Dive

12	Backend Code Structure
13	Database Design
14	Important APIs
15	Major Feature Flows
16	Exception Handling
17	Validation
18	Security
19	Caching
20	Notifications

PART IV Engineering Practice and Delivery

21	Docker
22	Deployment
23	CI/CD
24	Git Workflow
25	Development Challenges
26	My Contribution

PART V Design Decisions and Growth

27	Why This Technology
28	Alternative Technologies
29	Performance and Scalability
30	Future Improvements

PART VI Interview Question Bank

31	Project Interview Questions
32	Scenario-Based Questions
33	Core Java Questions
34	Spring Boot Questions
35	Spring Security Questions

36	JWT Questions
37	Database and SQL Questions
38	JPA and Hibernate Questions
39	React and Frontend Questions
40	Docker and Deployment Questions
41	Mock Interview Simulation

PART VII Revision Toolkit

42	Technical Glossary
43	Things Not to Say in an Interview
44	My Complete Project Story
45	One-Page Quick Revision
46	Top 20 Interview Questions
47	Final Interview Checklist

PART I

Project Foundations

01 Project at a Glance

BitInsight is a full-stack cryptocurrency trading and digital asset wallet web platform. Users can:

- Explore **live crypto market prices**
- Execute **real-time buy and sell orders**
- Manage an **integrated fiat/crypto wallet**
- Make **peer-to-peer (wallet-to-wallet) transfers**
- Deposit funds through **two payment gateways** (Razorpay and Stripe)
- Submit **bank withdrawal requests**
- Track portfolio holdings with **interactive charts**
- Secure their account with **Two-Factor Authentication (2FA)** via email OTP

Quick Project Metadata

Attribute	Value	Status
Project name	BitInsight (Crypto Trading & Wallet Platform)	✓
Architecture	Monolithic REST API backend + Single Page Application (SPA) frontend	✓
Backend framework	Spring Boot 3.4.5 on Java 21	✓
Database	MySQL (relational schema <code>yt_treading</code>)	✓
Frontend framework	React 19 + Vite 6 + Tailwind CSS + Redux	✓
Default backend port	5454 (from <code>application.properties</code>)	✓
Default frontend port	5173 / 5174 (from <code>AppConfig.java</code> and <code>CorsConfig.java</code>)	✓

✓ Remember

It is a **monolith**, not microservices, and it uses **Caffeine**, not Redis. Say this early and proudly. It shows you understand your own architecture.

02 One-Minute Pitch

🔥 Interview Priority

"Tell me about your project." is the first question in almost every interview. Practise this until it sounds like a conversation, not a recitation. Target: **60 to 90 seconds**.

Your spoken answer:

I built **BitInsight**, a full-stack cryptocurrency trading and digital wallet application. The goal was to give retail users a simple and secure place to trade crypto and manage their digital assets.

On the **backend**, I used **Java 21** and **Spring Boot 3.4.5**, with **Spring Security** and **JWT** for stateless authentication. The data layer uses **Spring Data JPA** and **Hibernate** with a **MySQL** database. I integrated the **CoinGecko REST API** for live prices and historical chart data. To avoid hitting its rate limits, I added an in-memory **Caffeine cache** and a Spring `@Scheduled` background task. For payments, I integrated **two gateways: Razorpay** for Indian Rupees and **Stripe** for international card payments in USD, so users can deposit money into their wallet.

On the **frontend**, I used **React 19**, **Vite**, **Tailwind CSS**, **Redux**, and **ApexCharts** for interactive price trends.

Users can register, enable Two-Factor Authentication through an email OTP, deposit funds, buy and sell crypto with transactional balance updates, transfer funds wallet-to-wallet, keep a watchlist, and submit withdrawal requests to their linked bank account.

My main contribution was the backend: the REST APIs, the transactional order and wallet services, the payment and CoinGecko integrations, the caching, and connecting everything to the React client.

Interview Tip

Structure to remember: **What it is → Backend → Frontend → What users can do → What you personally built.**

03 Detailed Walkthrough

Interview Tip

Use this version when the interviewer says "Can you describe the modules, architecture, and design decisions?"
Take about **2 to 3 minutes**.

Your spoken answer:

The application solves the problem of scattered crypto investing and wallet management. It combines live market data, fiat deposits and withdrawals, and spot trading in one platform.

Architecture Overview

The system uses a **decoupled client-server architecture**, in four layers:

Layer	What it does
1. Frontend	A React 19 SPA bundled with Vite . Redux Thunk manages authentication and coin data. Page-level workflows use Axios through a centralized API client that attaches the Bearer token automatically. The UI uses Tailwind CSS and Radix UI primitives.
2. Security and controller	Spring Boot REST controllers expose <code>/auth/**</code> (public authentication), <code>/coins/**</code> (public market data), and <code>/api/**</code> (protected trading and wallet operations). Requests to <code>/api/**</code> pass through the custom JwtTokenValidator filter. It extracts the token, verifies the HMAC-SHA signature, parses the claims, and populates the <code>SecurityContextHolder</code> .
3. Service and business	See the service breakdown below.

Layer	What it does
4. Data persistence	MySQL stores <code>User</code> , <code>Wallet</code> , <code>Asset</code> , <code>Order</code> , <code>OrderItem</code> , <code>Coin</code> , <code>Watchlist</code> , <code>PaymentOrder</code> , and <code>Withdrawal</code> . Relationships use JPA annotations such as <code>@OneToOne</code> , <code>@ManyToOne</code> , and <code>@ManyToMany</code> .

Service Layer Breakdown

Service	Responsibility
Order Service	Handles spot buying and selling inside a Spring <code>@Transactional</code> boundary. Asset balance changes and wallet deductions happen atomically . If any step fails, the whole transaction rolls back.
Wallet Service	Implements wallet ledger logic for deposits, order payments, and wallet-to-wallet transfers.
Coin Service and External Client	Calls CoinGecko through <code>RestTemplate</code> . To handle its aggressive HTTP 429 rate limits, I added a Caffeine cache with a 5-minute TTL and a <code>@Scheduled</code> task that refreshes the top coin lists every 10 minutes.
Payment Service	Creates Stripe checkout sessions and Razorpay payment links , then handles callback verification to credit the user's wallet.
User and 2FA Service	Supports email OTP verification through Spring Mail for password reset and two-factor login.

✓ Remember

Today the application runs locally: backend on port **5454**, Vite on **5173**, with CORS configured so the React client can call the API with an `Authorization` header.

04 Business Problem

What problem existed before this application?

#	Problem	Why it matters
1	High complexity for beginners	Established crypto exchanges are cluttered with derivatives, order books, and leverage tools that overwhelm newcomers.
2	Fragmented wallet and bank transfers	Beginners struggle to understand how fiat deposits become trading balances and how bank withdrawals work.
3	Weak security in simplified tools	Basic trading sandboxes often lack features like 2FA and password recovery.
4	API rate limiting	Market data APIs enforce strict limits. Fetching live prices on every user click quickly fails with HTTP 429 (Too Many Requests) .

How the project solves it

Solution	How it works
Unified fiat-to-crypto flow	Users deposit fiat through familiar gateways (Razorpay / Stripe) into an in-app wallet balance that can be used immediately to buy crypto.
Instant spot execution	Orders are calculated directly against live prices. The user's asset portfolio and wallet balance update atomically .
Built-in resilience	An in-memory Caffeine cache plus background cache warming keeps market responses fast, even under external API throttling.

Solution	How it works
Email OTP-based 2FA	One-time codes sent by email protect login and password reset.

05 Users and Roles

Roles are defined by the `USER_ROLE` enum (`com.bg.domain.USER_ROLE`) .

Role	Purpose
<code>ROLE_CUSTOMER</code>	Regular retail trader
<code>ROLE_ADMIN</code>	Platform administrator

Role and Permission Matrix

Role	Main permissions and access	What they cannot do
<code>ROLE_CUSTOMER</code>	- View live coins and market charts (<code>/coins/**</code>) - Register, log in, use 2FA, reset password (<code>/auth/**</code>) - View profile and toggle 2FA (<code>/api/users/**</code>) - Buy and sell crypto (<code>/api/orders/pay</code>) - View own orders and assets (<code>/api/orders</code> , <code>/api/asset</code>) - View and manage wallet, transfer wallet-to-wallet (<code>/api/wallet/**</code>) - Deposit via Stripe/Razorpay (<code>/api/payment/**</code> , <code>/api/wallet/deposit</code>) - Request withdrawal and save bank details (<code>/api/withdrawal/**</code> , <code>/api/payment-details</code>) - Manage watchlist (<code>/api/watchlist/**</code>)	- Approve or reject withdrawal requests - View all users' withdrawal requests
<code>ROLE_ADMIN</code>	- View all platform withdrawal requests (<code>/api/admin/withdrawal</code>) - Approve or reject withdrawals (<code>/api/admin/withdrawal/{id}/proceed/{accept}</code>) - Full customer trading access as well	Move funds out of another user's wallet without authentication

Technical Note on Role Enforcement

The URL rules in `AppConfig.java` are:

```
.requestMatchers("/auth/**").permitAll()
.requestMatchers("/api/**").authenticated()
.anyRequest().permitAll()
```

Admin routes live under `/api/admin/**`. Spring Security checks only that the caller is **authenticated**. It does **not** apply `.hasRole("ADMIN")` or `@PreAuthorize("hasRole('ADMIN')")`.

Interview Tip

Present this as an **observation you made yourself**, not as a confession. Say: "In production I would add `.requestMatchers("/api/admin/**").hasRole("ADMIN")` to enforce strict role-based access."

06 Technology Stack

Backend

Technology	Where used	Why used in this project 	Without it
Java 21	Backend language	LTS release with modern syntax, high performance, and long-term enterprise support.	No modern language features, and Spring Boot 3.4's baseline would not be met.
Spring Boot 3.4.5	Core framework	Rapid development, auto-configuration, dependency injection, and embedded Tomcat.	Hundreds of lines of boilerplate XML/Java config and manual servlet container setup.
Spring Security 6	Security layer	Declarative filter chains to protect endpoints, manage session policy, and intercept unauthorized requests.	Endpoints would be exposed, and you would write custom servlet filters for token parsing.
JWT 0.12.6 (<code>io.jsonwebtoken</code>)	Authentication	Creates and verifies signed JSON Web Tokens (HMAC-SHA).	You would need stateful HTTP sessions, which hinder horizontal scaling.
Spring Data JPA	Data access	<code>JpaRepository</code> interfaces with automatic method generation (e.g. <code>findById</code>).	Repetitive JDBC code: <code>PreparedStatement</code> and manual <code>ResultSet</code> mapping.
Hibernate 6	ORM engine	Maps Java entities to tables and manages object lifecycles.	You would hand-write <code>CREATE TABLE</code> , <code>INSERT</code> , <code>UPDATE</code> , and <code>JOIN</code> SQL for every entity.
MySQL (Connector/J)	Relational database	Reliable, ACID-compliant storage for users, wallets, balances, transactions, and orders.	Data would be lost on restart, and in-memory storage would not support concurrent financial data.
Caffeine Cache	In-memory caching	High-performance near-cache with a 5-minute TTL for CoinGecko data (<code>coinList</code> , <code>marketChart</code>).	CoinGecko would return HTTP 429 after only a few page views.
Spring Mail (<code>JavaMailSender</code>)	Notifications	Sends transactional emails containing OTPs for 2FA and password recovery.	Users could not receive verification codes, so 2FA and password reset would be impossible.
Razorpay Java SDK 1.4.3	Payments (India)	Creates INR payment links with automatic callback redirection to top up wallets.	Indian users could not deposit via UPI, NetBanking, or domestic cards.
Stripe Java SDK 20.62.0	Payments (global)	Creates USD Stripe Checkout sessions for card payments.	International users could not deposit via global cards.
CoinGecko REST API	External market data	Live prices, market cap ranks, 24-hour changes, search, and historical chart data.	No price feeds or market data at all.
Lombok	Developer utility	Generates getters, setters, constructors, <code>toString</code> , and SLF4J loggers at compile time (<code>@Data</code> , <code>@Slf4j</code>).	Classes cluttered with boilerplate.

Frontend

Technology	Where used	Why used in this project 	Without it
React 19	UI framework	Declarative, component-based UI with efficient Virtual DOM updates.	Server-rendered JSP/Thymeleaf pages or tedious vanilla DOM code.
Vite 6	Build tool and dev server	Very fast Hot Module Replacement and optimized ES-module bundling.	Slower startup and reloads, typical of legacy Webpack setups.

Technology	Where used	Why used in this project 	Without it
Redux and Redux-Thunk	State management	Central store for authentication (<code>auth</code>) and coin market (<code>coin</code>) state.	Prop drilling across many nested components and fragile state sync.
Tailwind CSS	Styling	Utility-first CSS for clean, responsive, dark-mode-first design.	Thousands of lines of custom CSS with possible class-name collisions.
Radix UI Primitives	UI components	Accessible, unstyled components (Dialog, Avatar, RadioGroup, ScrollArea) styled with Tailwind.	Building accessible modals and dropdowns from scratch.
ApexCharts	Data visualization	Interactive time-series charts for price fluctuations.	Static or text-only prices with no visual trends.
Axios	HTTP client	Promise-based client with interceptors and automatic <code>Authorization: Bearer</code> headers.	Manual <code>fetch()</code> setup, error handling, and headers on every call.

Common Mistake

Saying "Spring Data JPA is my database." The database is **MySQL**. **JPA** is a specification, **Hibernate** is the ORM implementation, and **Spring Data JPA** is the repository abstraction on top.

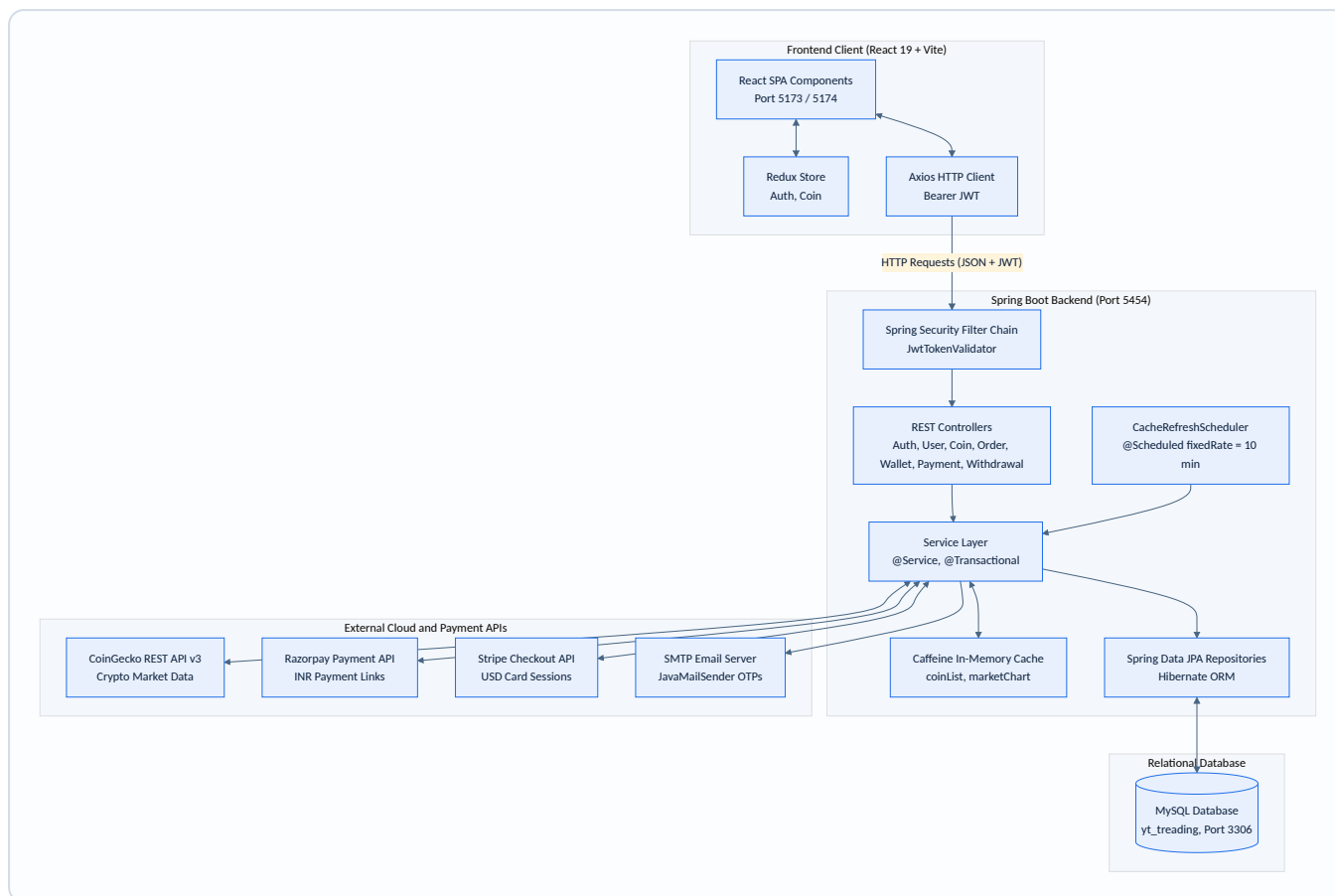
PART II

Architecture and Flow

07 System Architecture

BitInsight is a **monolithic client-server application** that integrates external SaaS services.

System Architecture Diagram



Component Details

#	Component	Role
1	React SPA client	Renders the dashboard, coin details, trading modals, wallet views, and admin withdrawal reviews. Talks to the backend over HTTP with CORS credentials enabled.
2	Spring Security and <code>JwtTokenValidator</code>	Intercepts requests, validates tokens for <code>/api/**</code> , and populates the <code>SecurityContext</code> with the authenticated email.
3	Service layer	Holds business logic, transactional balance accounting (<code>@Transactional</code>), and external API orchestration.
4	Caffeine cache	In-process cache (max 100 entries, 5-minute expiry) that shields the app from CoinGecko rate limits.

#	Component	Role
5	<code>CacheRefreshScheduler</code>	Background job every 10 minutes (<code>fixedRate = 600000</code> ms). Pre-fetches page 1 coins and top charts (Bitcoin, Ethereum, Tether).
6	MySQL	Stores core entity data with foreign-key relationships.
7	External APIs	CoinGecko supplies market data. Razorpay and Stripe generate payment links. SMTP delivers one-time passcodes.

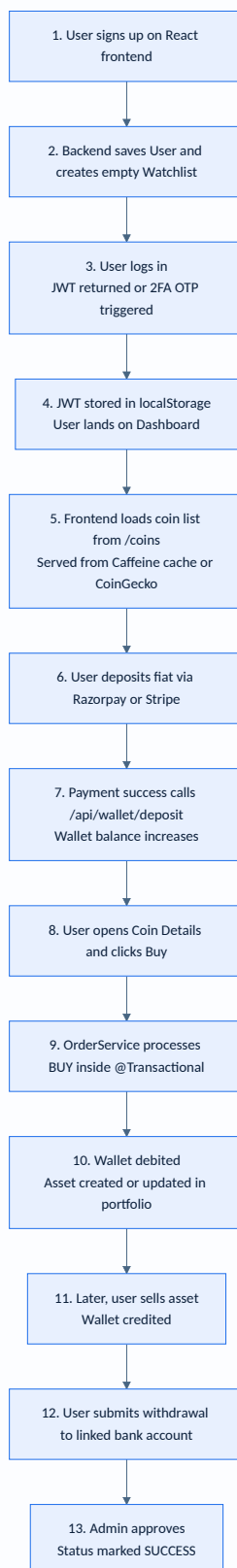


Key Concept

Monolith vs microservices. One deployable Spring Boot application with clear layers (controller → service → repository). This is simpler to build, test, and deploy at this project's scale than a set of distributed services.

08 End-to-End User Journey

A real user's path from signup to withdrawal:



Step-by-Step Breakdown

Step	Stage	What happens
1	Registration and setup	User submits email, password, and name to <code>/auth/signup</code> . The server checks email uniqueness, saves the user, generates a JWT, and creates an empty <code>Watchlist</code> .

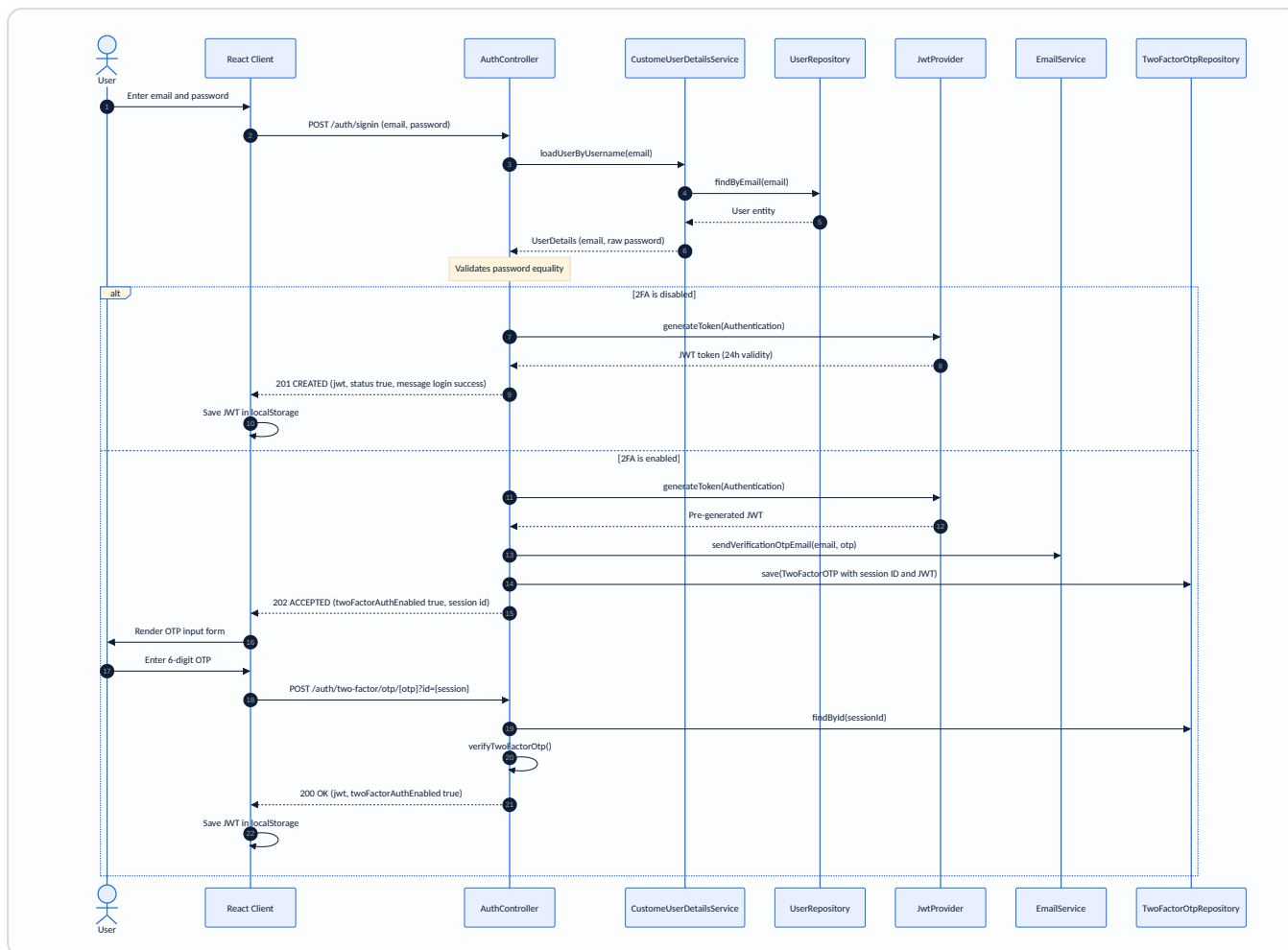
Step	Stage	What happens
2	Authentication and session	The JWT is returned and saved in browser <code>localStorage</code> . Axios attaches <code>Authorization: Bearer <jwt></code> to all later requests.
3	Market browsing	The frontend fetches top coins from <code>/coins</code> . If the data is in the Caffeine cache, it returns instantly without using CoinGecko API credits.
4	Wallet top-up	User picks an amount and Razorpay or Stripe. The backend calls the SDK, generates a checkout link, and redirects the user. After payment, the gateway redirects back to <code>/wallet?order_id=...</code> , which calls <code>/api/wallet/deposit</code> to credit the wallet.
5	Spot trading: Buy	User selects a quantity. The backend calculates the total (<code>price * quantity</code>), verifies the wallet balance, deducts it, creates an <code>Order</code> and <code>OrderItem</code> , and adds the coin to the user's <code>Asset</code> table.
5	Spot trading: Sell	The backend verifies the user owns enough quantity, creates a SELL order, decreases the asset balance (or deletes it if the balance drops below a threshold), and deposits the revenue into the wallet.
6	Withdrawal	User adds bank details (<code>/api/payment-details</code>) and requests a withdrawal (<code>/api/withdrawal/{amount}</code>). The amount is immediately deducted from the wallet and the status is <code>PENDING</code> . An admin approves it (<code>/api/admin/withdrawal/{id}/proceed/true</code>), moving it to <code>SUCCESS</code> .

09

Authentication Flow

Authentication is **stateless and token-based**. It supports email/password login with an optional **Two-Factor Authentication (2FA)** step.

Sequence Diagram: Authentication and 2FA



Detailed Mechanics

Stage	Details
Password comparison ✓	In <code>AuthController.java</code> , credentials are checked through <code>authenticate()</code> . <code>CustomUserDetailsService</code> loads the user by email, and the raw password strings are compared.
JWT creation ✓ (<code>JwtProvider.java</code>)	Algorithm: HMAC-SHA with a custom 256-bit secret key (<code>JwtConstant.java</code>). Claims: <code>email</code> and <code>authorities</code> (comma-separated roles). Expiry: issued-at + 86,400,000 ms (exactly 24 hours).
Validating later requests ✓ (<code>JwtTokenValidator.java</code>)	- Every request to <code>/api/**</code> is intercepted by this <code>OncePerRequestFilter</code>. - It reads the <code>Authorization</code> header. - It strips the <code>Bearer</code> prefix (first 7 characters). - It parses the claims with the HMAC key. - It extracts the email and authorities, creates a <code>UsernamePasswordAuthenticationToken</code>, and calls <code>SecurityContextHolder.getContext().setAuthentication(auth)</code>.

Key Concept

After a successful filter run, the `SecurityContextHolder` holds the current user for that request thread. Controllers then resolve the `User` from the JWT.

🔥 Interview Priority

Be ready to explain the login flow end to end: **credentials** → **CustomUserDetailsService** → **JwtProvider.generateToken()** → **JWT in localStorage** → **Authorization: Bearer** on every call → **JwtTokenValidator** → **SecurityContextHolder**.

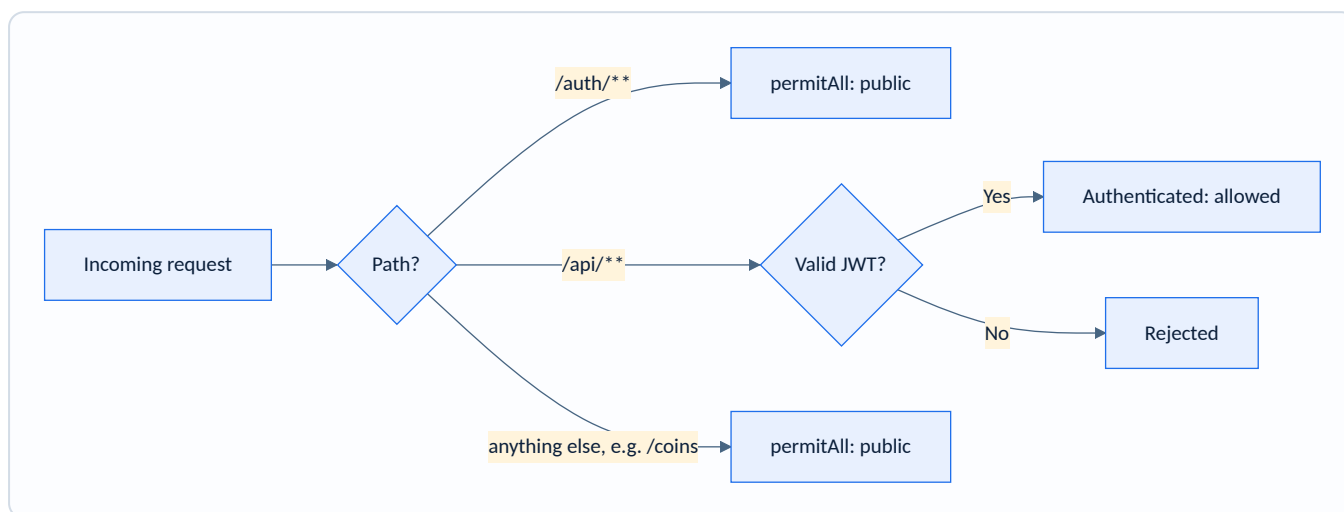
10 Authorization and RBAC**Authentication vs Authorization**

Concept	Question it answers	In BitInsight
Authentication (AuthN)	<i>Who are you?</i>	Validated at <code>/auth/signin</code> with email and password, then proven on later calls by the JWT signature.
Authorization (AuthZ)	<i>What are you allowed to do?</i>	Checks whether an authenticated identity may call a given endpoint.

Actual Implementation Status

Request matcher hierarchy in `AppConfig.java` :

Path pattern	Rule	Meaning
<code>/auth/**</code>	<code>permitAll()</code>	Public: login, signup, password reset
<code>/api/**</code>	<code>authenticated()</code>	Requires a valid JWT in the <code>Authorization</code> header
Anything else (e.g. <code>/coins</code>)	<code>permitAll()</code>	Public: coin list, charts, etc.

**Role-Based Access Control Analysis**

- Enums:** `USER_ROLE` contains `ROLE_CUSTOMER` and `ROLE_ADMIN`.
- Authority injection:** `JwtProvider` embeds the user's role in the JWT claim `authorities`. `JwtTokenValidator` converts it with `AuthorityUtils.commaSeparatedStringToAuthorityList(authorities)`.
- Current enforcement state:** 🟡 `/api/admin/**` (e.g. `/api/admin/withdrawal`) requires the caller to be **authenticated**, but there is no `.hasRole("ADMIN")` in `AppConfig` and no `@PreAuthorize`.
- What happens on an unauthorized call?**

- **No token or invalid token:** `JwtTokenValidator` throws `RuntimeException("invalid token")`, which results in HTTP **403 Forbidden** or **500 Internal Server Error**.
- **Authenticated non-admin calling `/api/admin/withdrawal`**: the request **succeeds** in the current build, because the role restriction is not bound to the route.

⚠ Common Mistake

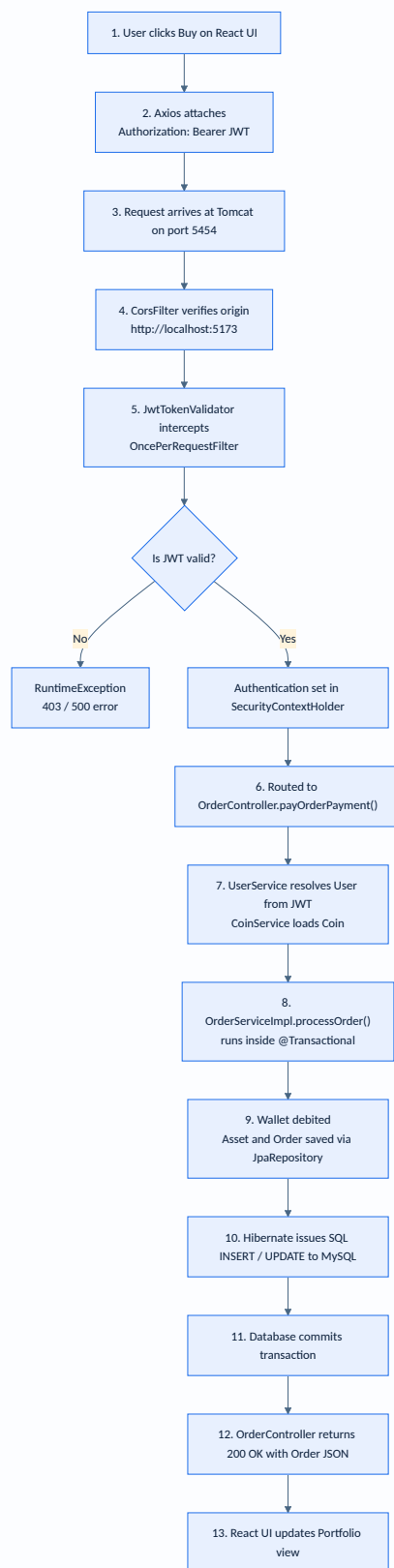
Claiming that admin endpoints are fully protected by roles. Say instead: *"Roles are stored and carried in the JWT, but admin-only enforcement is a next-step improvement."*

🎯 Interview Tip

Always highlight this gap and the fix yourself. It shows security awareness.

11 API Request Lifecycle

The lifecycle of an authenticated request, for example `POST /api/orders/pay` to buy Bitcoin:



✓ Remember

Order of the layers: **CORS** → **JWT filter** → **Controller** → **Service (@Transactional)** → **Repository** → **Hibernate** → **MySQL**. You can draw this on a whiteboard.

PART III

Implementation Deep Dive

12 Backend Code Structure

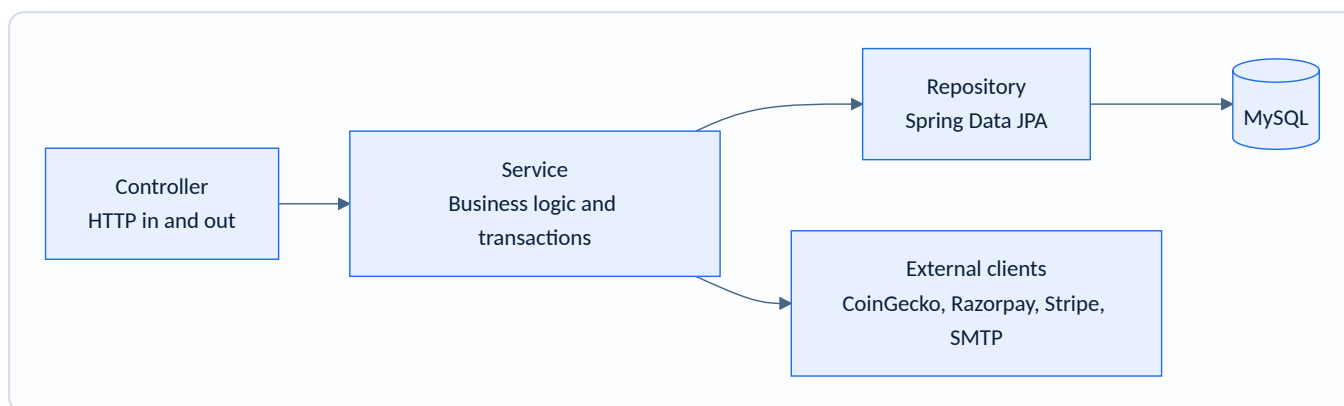
The backend follows standard layered Spring Boot conventions under the package `com.bg` :

```
com.bg
├─ client          # Empty / reserved package
├─ config          # Security, JWT, CORS, Caffeine cache, Global Exception Handler
├─ controller      # REST controllers handling incoming HTTP requests
├─ domain          # Enums (OrderStatus, OrderType, PaymentMethod, USER_ROLE, etc.)
├─ exception       # Reserved package for custom exceptions
├─ modal           # JPA entity classes (User, Wallet, Asset, Order, Coin, etc.)
├─ repository      # Spring Data JPA repository interfaces
├─ request         # Request DTOs (CreateOrderRequest, ResetPasswordRequest, etc.)
├─ response        # Response DTOs (AuthResponse, PaymentResponse, ApiResponse)
├─ scheduler       # CacheRefreshScheduler (background cache warming)
├─ service         # Service interfaces and implementations (*ServiceImpl)
└─ utils           # Utility classes (OtpUtils)
```

⚠ Common Mistake

Two names are spelled unusually in the codebase: the package `modal` (not `model`) and the class `CustomeUserDetailsService` (not `CustomUserDetailsService`). Use the exact spelling when you name them, and be ready to say "that's how it appears in my code."

Layer Responsibilities



Key Classes

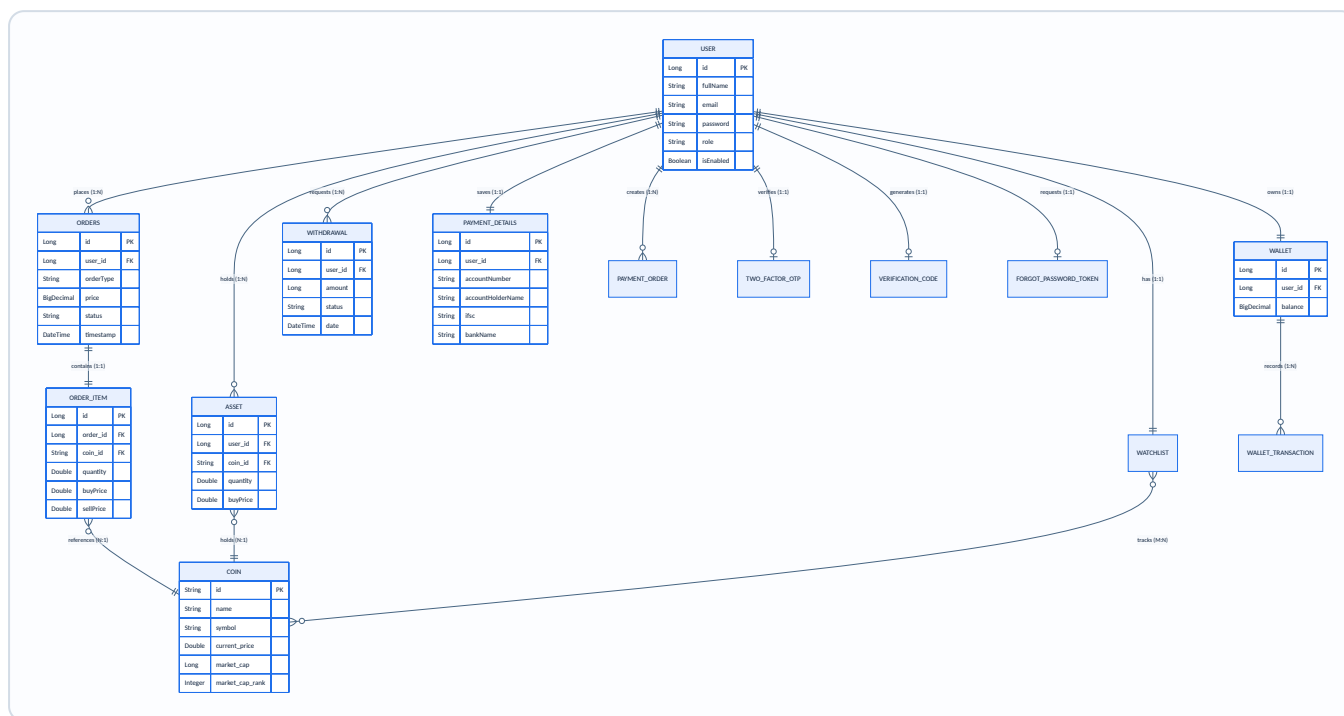
#	Class (package)	Purpose	Important methods and notes
1	AppConfig (<code>com.bg.config</code>)	Primary Spring Security and core bean configuration. Configures CORS origins, disables CSRF, sets stateless sessions, binds <code>JwtTokenValidator</code> , exposes a <code>RestTemplate</code> bean, and defines route authorization rules.	<code>securityFilterChain(HttpSecurity http)</code> , <code>corsConfigurationSource()</code> , <code>restTemplate()</code>
2	JwtProvider (<code>com.bg.config</code>)	Centralizes token signing and parsing with HMAC-SHA.	<code>generateToken(Authentication auth)</code> , <code>getEmailFromToken(String token)</code> . Called by <code>AuthController</code> and <code>UserServiceImpl</code> .
3	JwtTokenValidator (<code>com.bg.config</code>)	Custom filter extending <code>OncePerRequestFilter</code> . Runs before <code>BasicAuthenticationFilter</code> to extract and validate Bearer tokens and set <code>SecurityContextHolder</code> .	<code>doFilterInternal(HttpServletRequest request, HttpServletResponse, FilterChain)</code>
4	AuthController (<code>com.bg.controller</code>)	Public onboarding and authentication.	<code>register(@RequestBody User user)</code> : creates the user and an empty watchlist, returns a JWT. <code>login(@RequestBody User user)</code> : verifies credentials, returns a JWT or dispatches a 2FA OTP. <code>verifySignInOtp(@PathVariable String otp, @RequestParam String id)</code> : validates the 2FA OTP.
5	OrderController and OrderServiceImpl	Orchestrates crypto spot buying and selling.	<code>processOrder(Coin, quantity, OrderType, User)</code> : delegates to <code>buyAsset</code> or <code>sellAsset</code> . <code>buyAsset(...)</code> (<code>@Transactional</code>) : deducts wallet balance, records the order, creates or updates the asset. <code>sellAsset(...)</code> (<code>@Transactional</code>) : verifies ownership, records the order, deletes or decrements the asset, credits the wallet.
6	WalletController and WalletServiceImpl	Manages fiat balances, wallet-to-wallet transfers, and payment deposits.	<code>getUserWallet(User user)</code> : retrieves or initializes the wallet. <code>walletToWalletTransfer(User sender, Wallet receiver, Long amount)</code> : transfers with balance validation. <code>addBalance(Wallet wallet, Long money)</code> : adds or deducts funds.
7	CoinController and CoinServiceImpl	Public market data provider backed by CoinGecko.	<code>getCoinList(int page)</code> : top 10 market coins, cached via Caffeine. <code>getMarketChart(String coinId, int days)</code> : price and market cap history. <code>searchCoin(String query)</code> : search by symbol or name. <code>getTop50CoinsByMarketCapRank()</code> : top 50 global assets.

#	Class (package)	Purpose	Important methods and notes
8	PaymentController and PaymentServiceImpl	Razorpay and Stripe payment flows.	<code>createRazorpayPaymentLink(...)</code> : converts INR to paise, calls the Razorpay SDK, sets the callback URL. <code>createStripePaymentLink(...)</code> : creates a USD Stripe Checkout Session with success and cancel URLs. <code>ProceedPaymentOrder(PaymentOrder, String paymentId)</code> : verifies payment status with the Razorpay server.
9	CacheRefreshScheduler (<code>com.bg.service</code>)	Background worker with <code>@Scheduled(fixedRate = 600000)</code> (every 10 minutes). Pre-fetches page 1 market data and charts for Bitcoin, Ethereum, and Tether into the cache to avoid user-triggered cache misses and rate limits.	

13 Database Design

The schema is generated automatically by Hibernate (`spring.jpa.hibernate.ddl-auto=update`) against the MySQL database `yt_treading`. 

Entity-Relationship Diagram



Common Mistake

The diagram includes **TWO_FACTOR_OTP**, **VERIFICATION_CODE**, **FORGOT_PASSWORD_TOKEN**, and **WALLET_TRANSACTION**, but the table specification below does not describe them. If you are asked about their columns, **check your entity classes first** instead of guessing.

Table Specifications

Table	Primary key	Key columns	Relationships	Purpose
user	<code>id</code> (Long, auto)	<code>email</code> , <code>fullName</code> , <code>password</code> , <code>role</code> , <code>is_enabled</code> , <code>send_to</code>	1:1 with Wallet, Watchlist, PaymentDetails. 1:N with Order, Asset, Withdrawal.	User credentials, role, and embedded 2FA settings
wallet	<code>id</code> (Long, auto)	<code>balance</code> (BigDecimal), <code>user_id</code>	1:1 with User	Current fiat trading balance
asset	<code>id</code> (Long, auto)	<code>quantity</code> , <code>buyPrice</code> , <code>user_id</code> , <code>coin_id</code>	N:1 with User, N:1 with Coin	User's cryptocurrency holdings
coin	<code>id</code> (String, manual)	<code>name</code> , <code>symbol</code> , <code>current_price</code> , <code>market_cap</code> , <code>market_cap_rank</code>	Referenced by Asset, OrderItem, Watchlist	Cached snapshot of coin market data
orders	<code>id</code> (Long, auto)	<code>order_type</code> , <code>price</code> , <code>status</code> , <code>timestamp</code> , <code>user_id</code>	N:1 with User, 1:1 with OrderItem	Executed and pending buy/sell transactions
order_item	<code>id</code> (Long, auto)	<code>quantity</code> , <code>buy_price</code> , <code>sell_price</code> , <code>coin_id</code> , <code>order_id</code>	1:1 with Order, N:1 with Coin	Line-item details of the traded coin
watchlist	<code>id</code> (Long, auto)	<code>user_id</code>	1:1 with User, M:N with Coin	Favourite coins for quick monitoring
watchlist_coins	Composite	<code>watchlist_id</code> , <code>coins_id</code>	Join table: Watchlist ↔ Coin	Many-to-many coin bookmarking
payment_order	<code>id</code> (Long, auto)	<code>amount</code> , <code>payment_method</code> , <code>status</code> , <code>user_id</code>	N:1 with User	Fiat deposit checkout transactions
withdrawal	<code>id</code> (Long, auto)	<code>amount</code> , <code>status</code> , <code>date</code> , <code>user_id</code>	N:1 with User	Bank payout requests and approval status
payment_details	<code>id</code> (Long, auto)	<code>accountNumber</code> , <code>accountHolderName</code> , <code>ifsc</code> , <code>bankName</code> , <code>user_id</code>	1:1 with User	Bank account information for withdrawals

Interview Tip

Know the three core relationships cold: **User 1:1 Wallet**, **User 1:N Orders**, and **Order 1:1 OrderItem → N:1 Coin**.

14 Important APIs

Method	Endpoint	Purpose	Authentication	Role
POST	<code>/auth/signup</code>	Register a user and create a watchlist	Public	Anyone
POST	<code>/auth/signin</code>	Authenticate. Returns a JWT or prompts 2FA	Public	Anyone
POST	<code>/auth/two-factor/otp/{otp}</code>	Verify 2FA OTP and obtain a JWT	Public	Anyone
GET	<code>/coins</code>	Paginated cryptocurrency list	Public	Anyone
GET	<code>/coins/{coinId}/chart</code>	Price history chart coordinates	Public	Anyone

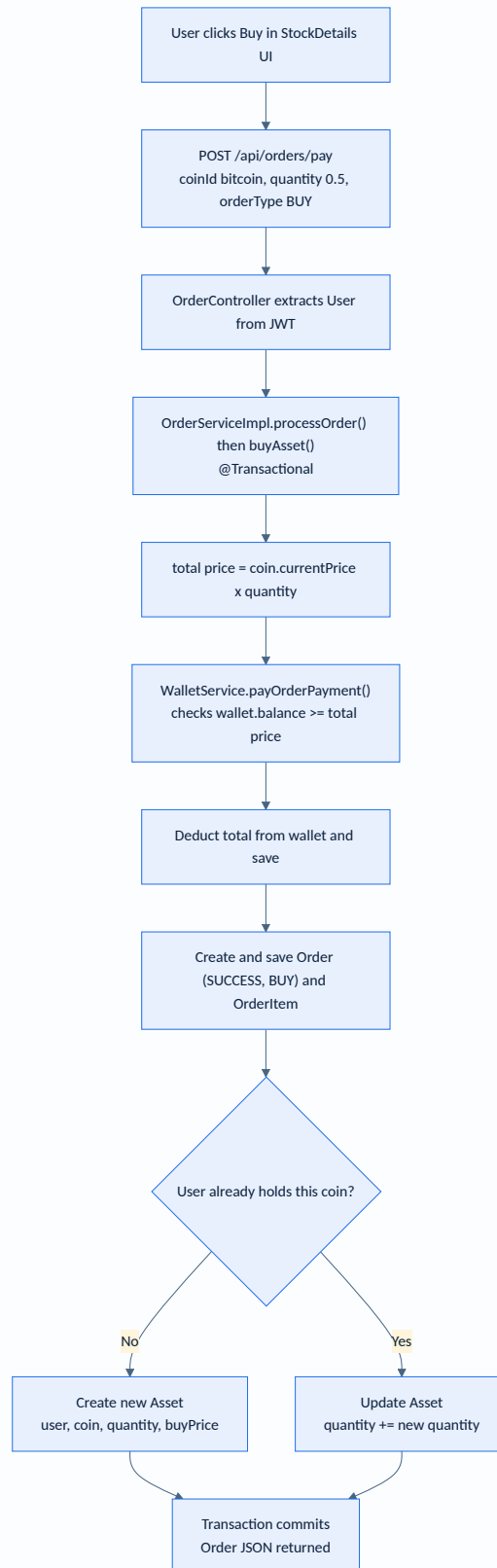
Method	Endpoint	Purpose	Authentication	Role
GET	/api/users/profile	Logged-in user profile	Bearer JWT	Authenticated
POST	/api/orders/pay	Place and execute a BUY or SELL order	Bearer JWT	Authenticated
GET	/api/orders	User's order execution history	Bearer JWT	Authenticated
GET	/api/wallet	Current wallet balance	Bearer JWT	Authenticated
PUT	/api/wallet/deposit	Confirm payment and credit the wallet	Bearer JWT	Authenticated
PUT	/api/wallet/{id}/transfer	Wallet-to-wallet transfer	Bearer JWT	Authenticated
POST	/api/payment/{method}/amount/{amount}	Start a Razorpay or Stripe payment	Bearer JWT	Authenticated
POST	/api/withdrawal/{amount}	Request a bank withdrawal	Bearer JWT	Authenticated
GET	/api/withdrawal	User's withdrawal history	Bearer JWT	Authenticated
GET	/api/admin/withdrawal	All platform withdrawal requests	Bearer JWT	Authenticated (admin by intent)
PATCH	/api/admin/withdrawal/{id}/proceed/{accept}	Approve or reject a withdrawal	Bearer JWT	Authenticated (admin by intent)
GET	/api/asset	Crypto assets in the user's portfolio	Bearer JWT	Authenticated
PATCH	/api/watchlist/add/coin/{coinId}	Add or toggle a coin in the watchlist	Bearer JWT	Authenticated

⚠ Common Mistake

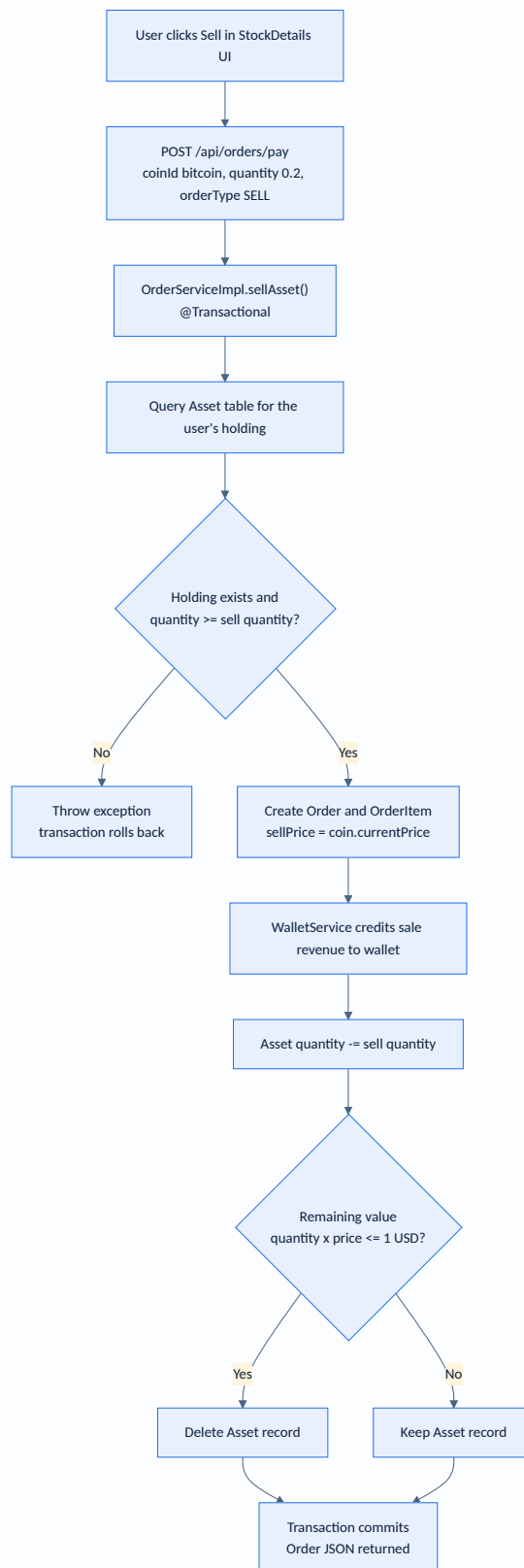
The two `/api/admin/...` routes are intended for admins, but as covered in [Section 10](#) the role is not yet enforced at the route level.

15 Major Feature Flows

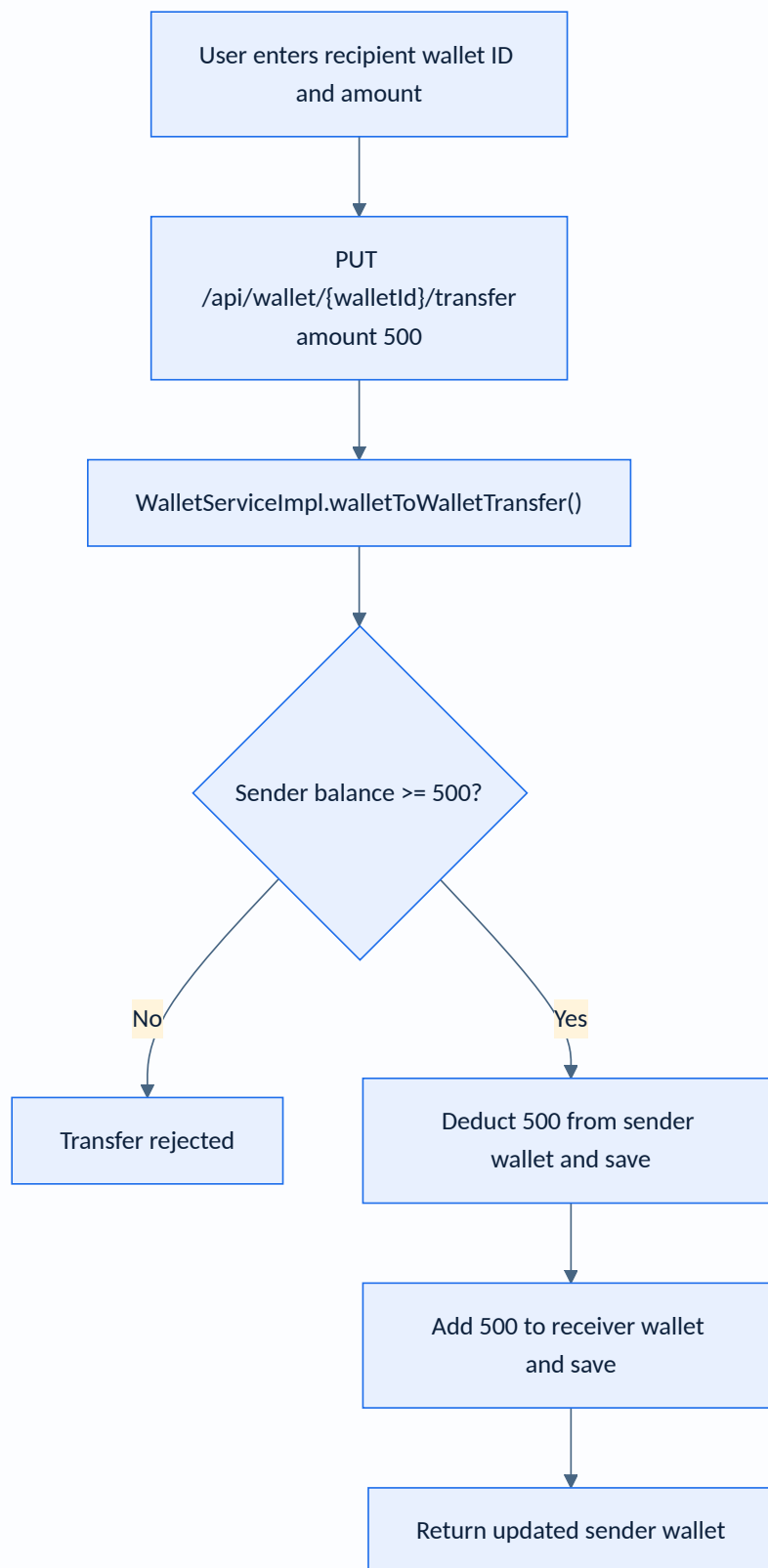
15.1 Spot Buy Crypto



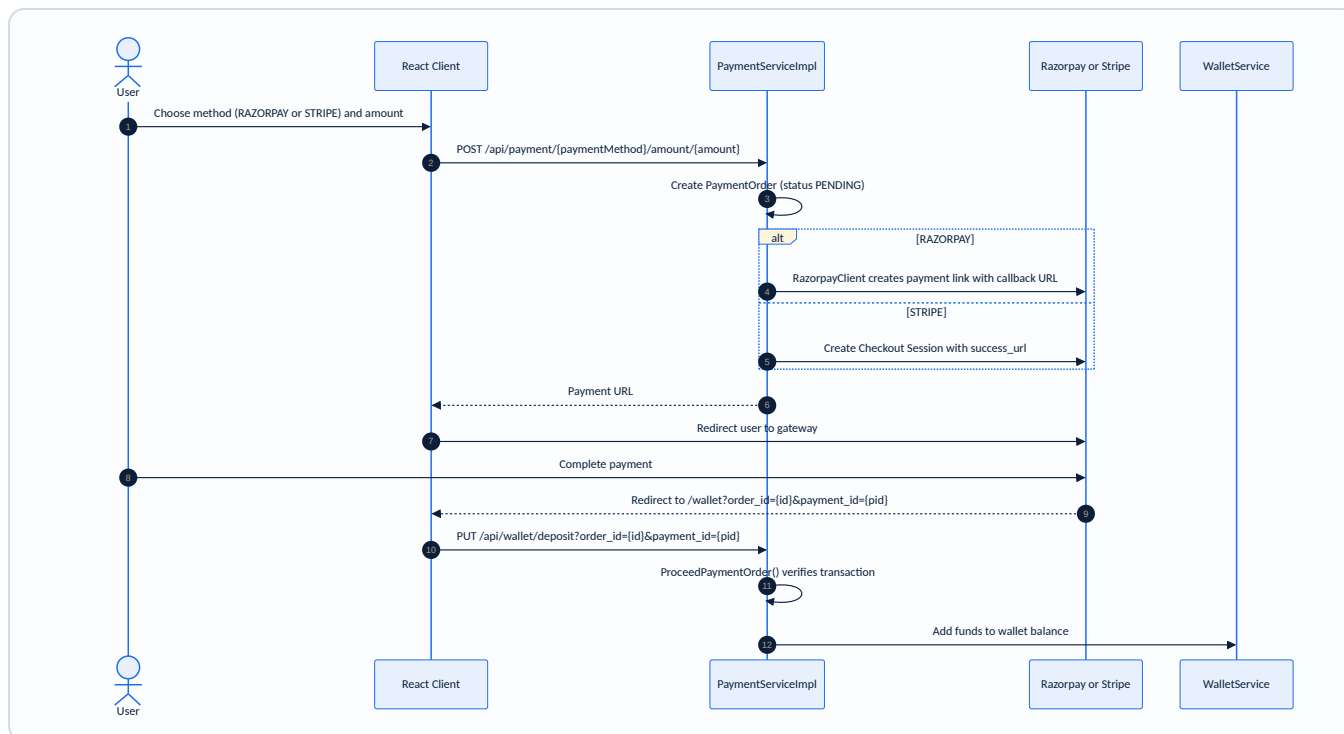
15.2 Spot Sell Crypto



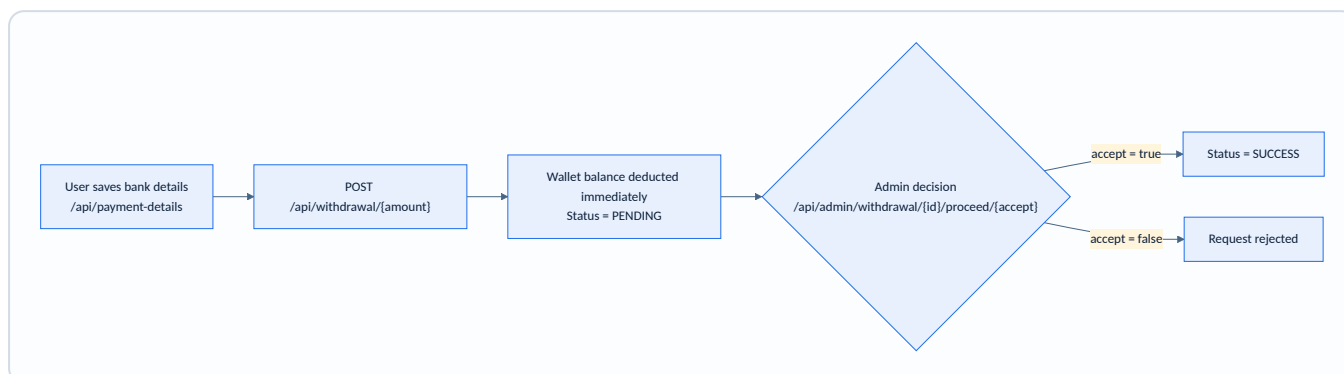
15.3 Wallet-to-Wallet Peer Transfer



15.4 Deposit Fiat Money (Razorpay / Stripe)



15.5 Withdrawal Lifecycle



🔥 Interview Priority

The **Buy flow** is the single most asked feature walkthrough. Be able to explain **why @Transactional matters**: wallet debit, order record, and asset update must all succeed together or all roll back.

⚠️ Common Mistake

The withdrawal deducts balance with `walletService.addBalance(userWallet, - withdrawal.getAmount())`, and a `wallet.getBalance() >= amount` check **should be enforced first** (see [Scenario 5](#)). Describe this as a known improvement area, not as existing behaviour.

16 Exception Handling

The application handles errors centrally through a `@ControllerAdvice` class.

GlobalExceptionHandler (com.bg.config)

```
@ControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(HttpClientErrorException.TooManyRequests.class)
    public ResponseEntity<ApiResponse> handleTooManyRequests(HttpClientErrorException.TooManyRequests
e) {
        log.error("Global Exception Handler: 429 Too Many Requests - {}", e.getMessage());
        ApiResponse response = new ApiResponse();
        response.setMessage("Rate limit exceeded. Please try again later. If data was previously
cached, it should be available.");
        return new ResponseEntity<>(response, HttpStatus.TOO_MANY_REQUESTS);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse> handleGeneralException(Exception e) {
        log.error("Global Exception Handler: General error - {}", e.getMessage());
        ApiResponse response = new ApiResponse();
        response.setMessage("An unexpected error occurred: " + e.getMessage());
        return new ResponseEntity<>(response, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

HTTP Status Codes Used

Code	Meaning	Where it appears in BitInsight
200 OK	Successful retrieval	Coins, orders, watchlist
201 CREATED	Resource created	User signup, payment link creation
202 ACCEPTED	Accepted for processing	2FA OTP dispatch, wallet balance adjustments
400 BAD REQUEST	Invalid parameters or validation failure	Bad input
403 FORBIDDEN	Invalid or missing JWT	Protected routes
429 TOO MANY REQUESTS	CoinGecko rate limit hit	Market data calls
500 INTERNAL SERVER ERROR	Unhandled runtime exceptions	Catch-all handler

Key Concept

`@ControllerAdvice` gives **one place** to convert exceptions into consistent `ApiResponse` JSON, instead of repeating `try/catch` in every controller.

17 Validation

Validation is applied in **three tiers**:

Tier	What is validated
1. Frontend	React components validate inputs (non-empty emails, password confirmation, positive transfer amounts) before calling the API.

Tier	What is validated
2. Backend service	<code>OrderServiceImpl</code> : checks <code>quantity > 0</code> , checks asset ownership before selling, verifies sufficient balance. <code>WalletServiceImpl</code> : checks <code>senderWallet.getBalance().compareTo(amount) >= 0</code> to prevent overdrafts. <code>AuthController</code> : checks whether the email already exists (<code>userRepository.findByEmail</code>).
3. Database constraints	<code>Order.java</code> : <code>@Column(nullable = false)</code> on <code>orderType</code> , <code>price</code> , and <code>status</code> . <code>User.java</code> : natural primary-key uniqueness on email.

18 Security

Implementation Status Matrix

Security feature	Status	Technical details (confirmed from code)
Stateless sessions	✅ Implemented	<code>SessionCreationPolicy.STATELESS</code> in <code>AppConfig.java</code> . No server-side HTTP sessions.
JWT authentication	✅ Implemented	JJWT 0.12.6 with an HMAC-SHA signing key. Tokens valid for 24 hours.
CORS protection	✅ Implemented	<code>AppConfig</code> and <code>CorsConfig</code> allow <code>http://localhost:5173</code> , <code>5174</code> , <code>3030</code> with <code>AllowCredentials(true)</code> .
CSRF protection	⦿ Disabled by design	<code>csrf.disable()</code> because the API uses stateless Bearer JWTs, not session cookies.
Two-factor auth (2FA)	✅ Implemented	Optional email OTP during sign-in using <code>TwoFactorOTP</code> and <code>EmailService</code> .
Password recovery	✅ Implemented	OTP-based reset through <code>/auth/users/reset-password/*</code> .
Password hashing	⦿ Partial: plain text ⚠️	<code>AuthController.java</code> saves the raw password (<code>newUser.setPassword(user.getPassword())</code>) and compares with <code>password.equals(...)</code> . No <code>PasswordEncoder</code> such as <code>BCryptPasswordEncoder</code> yet.
Role-based URLs	⦿ Partial	<code>AppConfig</code> enforces <code>.authenticated()</code> for <code>/api/**</code> but has not bound <code>.hasRole("ADMIN")</code> to <code>/api/admin/**</code> .
Refresh tokens	🔮 Future	A single 24-hour access token is used. No refresh-token rotation.

🎯 Interview Tip

Explaining that passwords are currently stored in plain text, and *exactly how you would fix it* with `BCryptPasswordEncoder` in the security configuration, shows real security awareness. Interviewers value an honest, specific plan far more than a false claim.

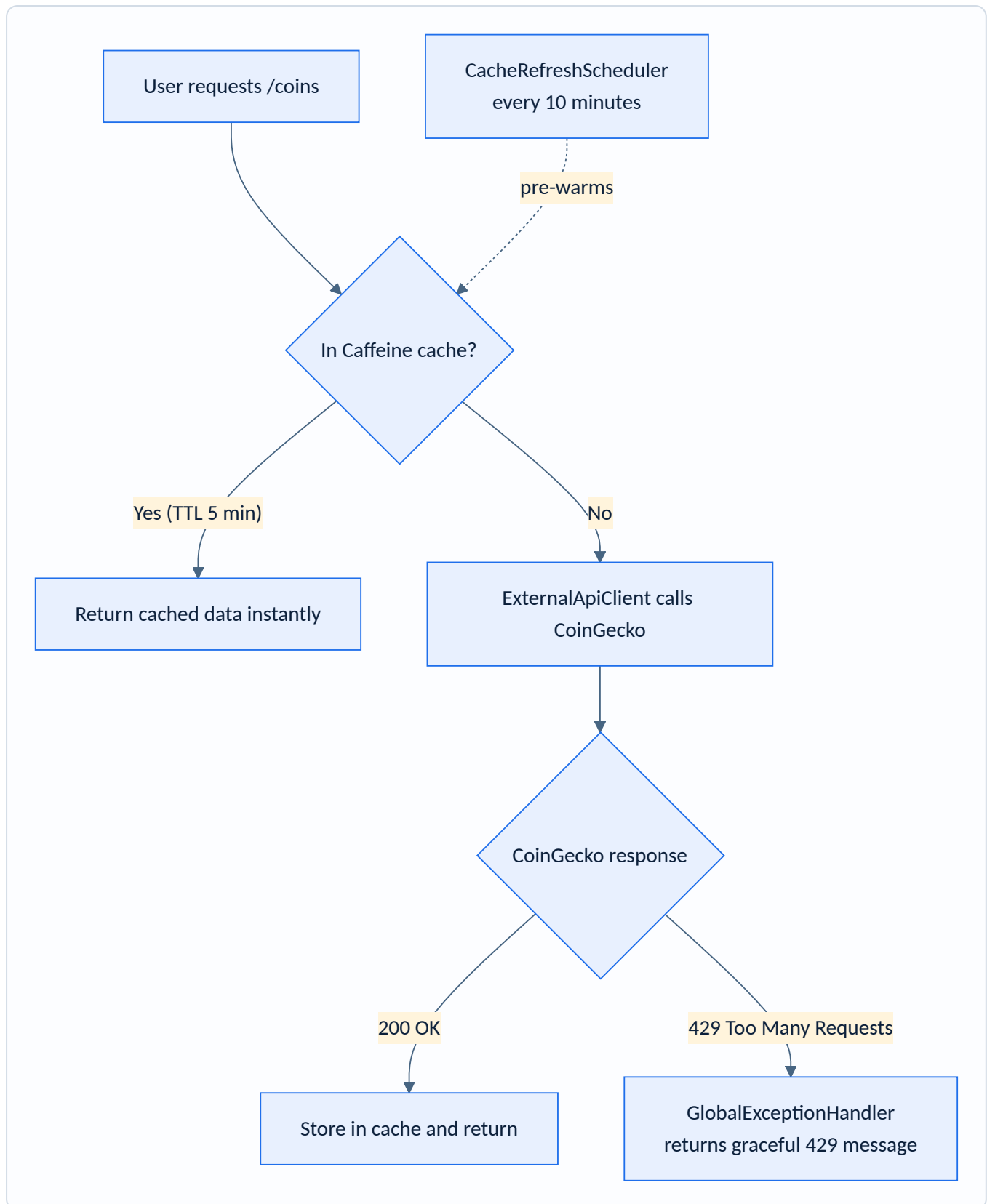
19 Caching

Redis vs Caffeine: Clarification

Technology	Status	Evidence
Redis	 Not implemented	Not present in <code>pom.xml</code>
Caffeine in-memory cache	 Implemented	<code>CacheConfig.java</code>

How Caffeine Works in This Project

Part	Details
Configuration (<code>CacheConfig.java</code>)	Two caches: <code>coinList</code> and <code>marketChart</code> . Initial capacity 10 , maximum size 100 . Eviction: <code>expireAfterWrite(5, TimeUnit.MINUTES)</code> . Metrics: <code>.recordStats()</code> enabled.
Proactive refresh (<code>CacheRefreshScheduler.java</code>)	<code>@Scheduled(fixedRate = 600000)</code> , every 10 minutes. Pre-loads the page 1 coin list and charts for <code>bitcoin</code> , <code>ethereum</code> , and <code>tether</code> .
If the cache expires or fails	The app falls back to <code>ExternalApiClient</code> to fetch directly from CoinGecko. On HTTP 429, <code>GlobalExceptionHandler</code> catches it and informs the user gracefully.



🔥 Interview Priority

"How did you solve CoinGecko's rate limiting?" Answer with the trio: **Caffeine cache (5-minute TTL) + @Scheduled refresh every 10 minutes + graceful 429 handling.**

20 Notifications

Channel	Status	Details
Email	 Implemented	Spring Boot Mail (<code>spring-boot-starter-mail</code>) and <code>JavaMailSender</code> (<code>EmailService.java</code>). Sends one-time passcodes for 2FA and password reset.
Push (Firebase FCM)	 Not implemented	
SMS	 Not implemented	

PART IV

Engineering Practice and Delivery

21 Docker

Status

Current repository  Not implemented. No `Dockerfile` or `docker-compose.yml` exists in the codebase.

Production Implementation Plan (for interview discussion)

Component	Plan
Backend Dockerfile	Multi-stage build using <code>eclipse-temurin:21-jdk-alpine</code> to compile the Maven artifact, then run the <code>.jar</code> inside a lightweight JRE container.
Frontend Dockerfile	Multi-stage build using <code>node:20-alpine</code> to run <code>npm run build</code> , followed by an <code>nginx:alpine</code> image to serve the static HTML/JS assets.
Docker Compose	Orchestrates three containers: <code>mysql:8.0</code> , <code>backend</code> (port 5454), and <code>frontend</code> (port 80) on a private bridge network.

 Common Mistake

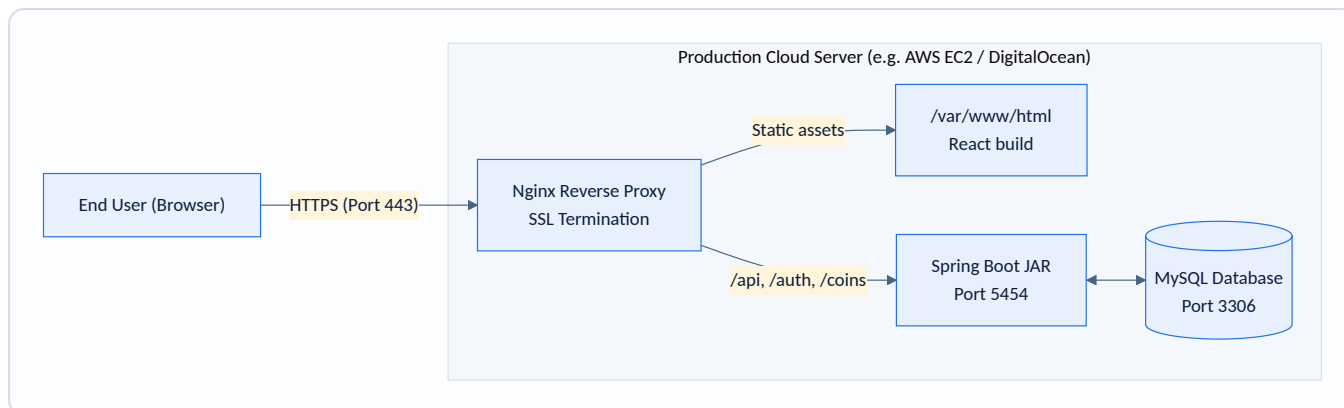
Saying "I deployed this with Docker." Say: "It runs locally today. I have planned how I would containerize it with multi-stage Dockerfiles and Docker Compose."

22 Deployment

Current Local Deployment

```
+-----+
| Local Developer Machine (Windows) |
|                                     |
| 1. MySQL Server running locally on localhost:3306 (Database: yt_treading) |
| 2. Spring Boot Tomcat application running on localhost:5454 |
| 3. React Vite development server running on localhost:5173 / 5174 |
+-----+
```

Target Production Architecture 🧙



23 CI/CD

Status

Current 🧙 Not implemented. No GitHub Actions workflow files exist in `.github/workflows/`.

Target Pipeline (for interview discussion) 🧙



Step	Stage	Action
1	Trigger	<code>git push origin main</code> or pull request creation
2	CI: backend	Set up JDK 21 and run <code>mvn clean verify</code> to execute automated tests
3	CI: frontend	Set up Node.js 20, install with <code>npm ci</code> , run <code>npm run build</code> and <code>npm run lint</code>
4	CD: images	Build Docker images and push to Docker Hub or AWS ECR
5	CD: deploy	SSH into the production server and run <code>docker compose pull && docker compose up -d</code>

24 Git Workflow

Based on repository inspection ✅:

Item	Detail
Active feature branch	<code>Balaram_Treading_Application</code>

Item	Detail
Workflow style	Feature-branch development with incremental commits that track full-stack feature delivery

Recent Commits

Commit	Message
73fcda b	feat: add two-factor authentication login flow, enhance portfolio and activity components, and improve wallet management
82055d 1	feat: implement account verification, order details, and payment confirmation components
836ae5 7	feat: add UI components for card, dialog, form, input OTP, label, pagination, radio group, scroll area, and wallet features
086247 d	Add cors config in my backend

25 Development Challenges

Interview Tip

Use the **Problem** → **Solution** → **Lesson** format. Interviewers love a clear engineering story. These three challenges map directly to your code.

Challenge 1: Third-Party API Rate Limiting (CoinGecko HTTP 429)

Problem	CoinGecko's free tier has a strict rate limit (10 to 30 requests per minute). Moving quickly between dashboard pages caused HTTP 429 errors.
Solution	Configured an in-memory Caffeine cache (CacheConfig.java) with a 5-minute TTL. Added CacheRefreshScheduler with @Scheduled tasks to pre-fetch top coin lists and charts every 10 minutes.
Lesson	Never call an external API synchronously on every user read. Caching and background polling decouple your availability from third-party rate limits.

Challenge 2: Transactional Integrity in Spot Order Execution

Problem	A buy involves three database changes: deduct wallet funds, create an order record, create or update an asset. If the server crashed after deducting the wallet but before saving the asset, user funds would be lost.
Solution	Wrapped the execution in Spring's @Transactional in OrderServiceImpl.java . If asset creation fails, the wallet deduction rolls back automatically.
Lesson	Financial operations need strict ACID boundaries to prevent ledger inconsistencies.

Challenge 3: CORS with a Custom Authorization Header

Problem	The browser blocked requests from Vite (localhost:5173) to Spring Boot (localhost:5454) with CORS errors when sending the Authorization header.
Solution	Configured CorsConfigurationSource in AppConfig.java to allow credentials, permit GET , POST , PUT , PATCH , DELETE , and expose the Authorization header.

Lesson Modern browsers need explicit preflight ([OPTIONS](#)) handling and header-exposure configuration.

26 My Contribution

Clearly Visible From Repository Code

#	Area	What you did
1	Spring Boot backend architecture	Designed and implemented controllers, service interfaces, service implementations, and JPA entities across all modules: Auth , User , Order , Wallet , Coin , Payment , Withdrawal , Watchlist .
2	Security and authentication	Implemented JwtTokenValidator , JwtProvider , and custom 2FA login verification with email OTPs.
3	Third-party integrations	Integrated CoinGecko REST APIs via RestTemplate , the Stripe Checkout SDK, and the Razorpay Java SDK.
4	Caching strategy	Configured the Caffeine cache manager and scheduled cache refresh tasks.
5	React frontend	Built interactive UI pages, Redux actions/reducers for Auth and Coins, the Axios API client, and ApexCharts market visualization.

Interview Contribution Statement

I developed the full-stack architecture of BitInsight. I built the Spring Boot REST backend, set up transactional order and wallet balance management, implemented JWT and Two-Factor Authentication, integrated CoinGecko, Stripe, and Razorpay, and built the responsive React user interface.

PART V

Design Decisions and Growth

27 Why This Technology

Inferred

The reasoning below is architectural rationale derived from the code patterns. Use it to explain *why*, but do not present it as a documented project decision record.

Question	Answer
Why Spring Boot instead of Spring MVC?	Spring Boot provides opinionated auto-configuration, starter dependencies, and an embedded Tomcat server. In traditional Spring MVC you configure <code>web.xml</code> , <code>DispatcherServlet</code> , view resolvers, and an external Tomcat manually.
Why MySQL instead of MongoDB?	Trading and wallet applications need strict ACID compliance, transactional integrity (<code>@Transactional</code>), and relational foreign keys between users, wallets, orders, and assets. A relational database prevents orphan records and phantom balance updates.
Why Caffeine instead of Redis?	For a single-instance monolith, Caffeine gives near-instant in-memory performance without the operational complexity and memory overhead of running an external Redis cluster.
Why JWT instead of HTTP sessions?	JWTs are stateless. The server does not allocate session memory per user. The token carries the claims and the signature, which simplifies horizontal scaling behind a load balancer.

28 Alternative Technologies

Used	Alternative	Key differences	When to choose the alternative
JWT	Stateful session cookies	Sessions keep state in server memory. JWT keeps verified claims in the client token.	Simple monoliths needing immediate server-side revocation on every click.
MySQL	PostgreSQL	Both are ACID relational databases. PostgreSQL has stronger JSON querying and custom data types.	Heavy use of JSONB or advanced geospatial/window functions.
Spring Data JPA	MyBatis / JDBC	JPA automates object mapping. MyBatis gives granular control over manual SQL.	Complex legacy databases with highly optimized DBA-written SQL (also JOOQ).
Caffeine	Redis	Caffeine is in-process (JVM memory). Redis is a distributed, out-of-process cache server.	Multiple clustered backend instances that must share one cache.
REST	GraphQL	REST exposes fixed endpoints per resource. GraphQL lets clients request exact fields in one call.	Diverse, deeply nested client data needs with mobile bandwidth constraints.

29 Performance and Scalability

Current Performance Characteristics

- Fast reads of the cryptocurrency market list thanks to **Caffeine caching**.
- Direct database writes with indexing on primary keys (`Long id`) and foreign keys.

Potential Bottlenecks and Optimizations 🔍

#	Area	Recommendation
1	Database indexing	Make sure indexes exist on frequently queried foreign keys such as <code>order.user_id</code> , <code>asset.user_id</code> , and <code>wallet.user_id</code> .
2	Connection pooling	Spring Boot 3 uses HikariCP by default, which provides fast connection pooling to MySQL.
3	Pagination	The CoinGecko integration and <code>/api/orders</code> should use Spring Data Pageable (<code>PageRequest.of(page, size)</code>) to avoid loading thousands of rows into memory.

30 Future Improvements

🎯 Interview Tip

End questions like "What would you improve?" with this list, ordered by impact. It turns weaknesses into a clear roadmap.

Priority	Improvement	What to do
1	🔒 Password encryption	Integrate <code>BCryptPasswordEncoder</code> in the security configuration to replace plain-text storage.
2	🛡️ Granular RBAC enforcement	Bind <code>.requestMatchers("/api/admin/**").hasRole("ADMIN")</code> so non-admins cannot reach administrative endpoints.
3	💡 Distributed caching (Redis)	Move from in-memory Caffeine to Redis when scaling to multiple backend instances.
4	📡 WebSocket live price feeds	Replace REST polling with WebSocket (STOMP) connections that stream live ticker prices to the React UI.
5	🧪 Automated testing and CI/CD	Add JUnit 5 and Mockito tests, plus a GitHub Actions pipeline for continuous integration and deployment.

PART VI

Interview Question Bank

Interview Tip

Questions marked  **High Priority** are the ones you should be able to answer **without notes**. For each project question, practise the *Short answer* first, then the *Full answer*, and keep the *Follow-up* ready.

31 Project Interview Questions

Category 1: Project Understanding

Q1. Can you explain what your project does?

- **Short answer:** It's a full-stack crypto trading web app. Users can track live prices, buy and sell assets using a fiat wallet, deposit money through Stripe or Razorpay, and transfer funds between wallets.
- **Full answer:** The platform combines market data from CoinGecko with internal spot trading and wallet ledger management. The frontend is React 19 and the backend is a Spring Boot 3.4 REST API secured with JWT and 2FA, backed by MySQL.
- **Follow-up:** *How do you make sure users can't buy crypto without enough balance?* **Answer:** In `OrderServiceImpl.java`, the order runs inside a `@Transactional` method. It checks that `wallet.balance >= totalOrderCost` before deducting. If the balance is insufficient, an exception is thrown and the order is aborted.

Q2. What was your specific role in this project?

- **Short answer:** I designed and built the backend REST APIs, implemented JWT and 2FA security, integrated the payment gateways and market data API, and developed the React UI.
- **Full answer:** I was responsible for the end-to-end implementation: database entity relationships, transactional business logic in the service layer, Caffeine caching, and the client-side UI with Redux and ApexCharts.
- **Follow-up:** *Which module was the most complex?* **Answer:** The `OrderService` and `WalletService` interaction. A spot order needs several database updates (wallet balance, order log, asset inventory) that must all succeed or all fail together.

Q3. How does your system architecture look?

- **Short answer:** It's a decoupled monolithic client-server architecture: a React SPA on the frontend and a Spring Boot REST API on the backend, talking to MySQL.
- **Full answer:** The frontend calls the backend over HTTP using Axios with Bearer tokens. Spring Security validates the token through a custom filter. The service layer uses Spring Data JPA for MySQL and talks to the external services (CoinGecko, Razorpay, Stripe, SMTP).
- **Follow-up:** *Why didn't you build this as microservices?* **Answer:** At this project's scale, a well-structured modular monolith is simpler to develop, test, and deploy. It avoids the overhead of service discovery, distributed tracing, and network latency.

Category 2: Core Java and Spring Boot

Q4. 🔥 Why did you choose Java 21 for this project?

- **Short answer:** Java 21 is a modern Long-Term Support (LTS) release with strong performance, pattern matching, record patterns, and Virtual Threads.
- **Full answer:** It's the LTS baseline supported by Spring Boot 3.x. It brings performance improvements, modern syntax, and future-ready concurrency features.
- **Follow-up:** What's the difference between `@RestController` and `@Controller`? **Answer:** `@Controller` is used in traditional MVC to return view templates. `@RestController` combines `@Controller` and `@ResponseBody`, so return values are automatically serialized to JSON/XML.

Q5. What does `@Transactional` do in your `OrderServiceImpl`?

- **Short answer:** It makes all database operations in the method run as one atomic unit. If an error occurs, every change rolls back.
- **Full answer:** Spring wraps the bean in a proxy using Spring AOP. It opens a database transaction, runs the method, and commits if no unhandled runtime exception occurs. If one is thrown, it rolls back to prevent inconsistent state.
- **Follow-up:** What happens if a checked exception is thrown inside a `@Transactional` method? **Answer:** By default Spring rolls back only on unchecked exceptions (`RuntimeException` and `Error`). To roll back on checked exceptions too, use `@Transactional(rollbackFor = Exception.class)`.

Category 3: Spring Security and JWT

Q6. 🔥 Explain your login and JWT generation flow.

- **Short answer:** The user sends credentials to `/auth/signin`. The server validates them, generates a signed JWT containing the user's email and roles, and returns it.
- **Full answer:** In `AuthController`, `authenticate()` loads the user through `CustomUserDetailsService` and checks the credentials. If they're valid, `JwtProvider.generateToken()` signs a token with HMAC-SHA that carries the claims and a 24-hour expiry. The client stores it in `localStorage`.
- **Follow-up:** How does the server validate the JWT on later requests? **Answer:** `JwtTokenValidator` intercepts requests to `/api/**`, reads the token from the `Authorization` header, parses the claims with the secret key, and puts an authenticated token into `SecurityContextHolder`.

Q7. 🔥 What is the purpose of `OncePerRequestFilter`?

- **Short answer:** It guarantees the filter runs exactly once per incoming HTTP request.
- **Full answer:** In servlet environments with internal forwards or dispatches, a normal filter can execute more than once. `OncePerRequestFilter` ensures a single execution per request dispatch, which is ideal for parsing an authentication token.
- **Follow-up:** Where in the filter chain is `JwtTokenValidator` placed? **Answer:** In `AppConfig.java`, it is registered with `.addFilterBefore(new JwtTokenValidator(), BasicAuthenticationFilter.class)`.

Category 4: Database and Hibernate / JPA

Q8. 🔥 Explain the relationship between `User`, `Order`, and `OrderItem`.

- **Short answer:** A User has many Orders (`@OneToMany`). Each Order has exactly one OrderItem (`@OneToOne`), and each OrderItem references a Coin (`@ManyToOne`).
- **Full answer:** `Order.java` has a `@ManyToOne` mapping to `User`. The Order-to-OrderItem link is a bidirectional `@OneToOne(mappedBy = "order", cascade = CascadeType.ALL)`. This separates order-level metadata from line-item pricing.
- **Follow-up:** What does `cascade = CascadeType.ALL` mean? **Answer:** Any persistence operation on the parent (save, update, delete) is automatically applied to the associated child.

Q9. What's the difference between Lazy and Eager loading in JPA?

- **Short answer:** Eager loading fetches associated entities immediately with the parent. Lazy loading fetches them only when your code accesses them.
- **Full answer:** Eager loading can cause serious performance problems (the N+1 query problem). Lazy loading uses Hibernate proxies that run extra SQL only when a getter is called inside an active transaction session.
- **Follow-up:** What is a `LazyInitializationException`? **Answer:** It happens when code accesses a lazily loaded relationship outside an active Hibernate Session or transaction.

Category 5: Caching and Rate Limiting

Q10. 🔥 How did you solve the CoinGecko rate-limiting problem?

- **Short answer:** I used a Caffeine in-memory cache with a 5-minute TTL and a background scheduler that pre-fetches popular coin data every 10 minutes.
- **Full answer:** Calling CoinGecko directly from client requests caused HTTP 429 errors. With a Caffeine cache (`CacheConfig.java`) and a `@Scheduled` task (`CacheRefreshScheduler.java`), user requests are served from memory, which insulates the system from the rate limit.
- **Follow-up:** Why Caffeine instead of Redis? **Answer:** Caffeine runs inside the JVM heap with zero network overhead, so it's ideal for a single-instance deployment that doesn't need external infrastructure.

32 Scenario-Based Questions

Inferred

Scenarios 2 to 5 describe behaviour you would reason about from the code and framework defaults. Frame them as "here's what would happen and how I'd harden it".

#	Scenario	Answer
1	A user sends an expired JWT. What happens?	When <code>JwtTokenValidator</code> parses the token with <code>Jwts.parser()</code> , the library throws an <code>ExpiredJwtException</code> . The filter catches it and throws <code>RuntimeException("invalid token")</code> , so the request does not proceed and an error response is returned.
2	Two users try to buy the last remaining quantity of a coin at the same time. How do you prevent race conditions?	Right now, crypto supply in this app is virtual. If inventory were constrained, I'd use optimistic locking (<code>@Version</code> on the entity) or pessimistic locking (<code>PessimisticLockMode.PESSIMISTIC_WRITE</code>) so only one transaction commits.

#	Scenario	Answer
3	The database becomes unavailable while an order is processing. What happens?	The JPA operation throws a <code>CannotCreateTransactionException</code> or <code>JDBCConnectionException</code> . Because the method is <code>@Transactional</code> , no partial changes are committed, and <code>GlobalExceptionHandler</code> returns HTTP 500.
4	The CoinGecko API goes completely down. How does the app behave?	If the requested data is already in Caffeine, users keep seeing cached prices. On a cache miss, <code>ExternalApiClient</code> throws an exception and <code>GlobalExceptionHandler</code> returns a graceful error saying external market data is temporarily unavailable.
5	A user requests a withdrawal larger than their balance. What happens?	In <code>WithdrawalController</code> , the current code deducts balance with <code>walletService.addBalance(userWallet, -withdrawal.getAmount())</code> . To prevent overdrafts, a check that <code>wallet.getBalance() >= amount</code> must be enforced before saving the withdrawal request.

⚠ Common Mistake

In Scenario 5, don't claim the withdrawal path already blocks overdrafts. State clearly that the validation is the improvement.

33 Core Java Questions

Q1. What is the difference between `==` and `.equals()` ?

- Short answer:** `==` compares references (whether two variables point to the exact same object). `.equals()` compares the actual content or values.
- Project link:** When comparing OTP strings in `TwoFactorOtpServiceImpl`, we use `.equals()` (`twoFactorOTP.getOtp().equals(otp)`), because we want matching characters, not the same memory location.

Q2. Why is `String` immutable in Java?

- Short answer:** Once a `String` is created, its value can never change. Modifying it creates a new `String` object.
- Why it matters:**
 - Security:** parameters such as database URLs, usernames, passwords, and network connections can't be maliciously altered.
 - Thread safety:** multiple threads can read the same string without synchronization problems.
 - String constant pool:** the JVM can cache and reuse strings, saving heap memory.

Q3. What is the difference between `ArrayList` and `LinkedList` ?

	<code>ArrayList</code>	<code>LinkedList</code>
Internal structure	Dynamic resizable array	Doubly-linked list (each node points to next and previous)
Search / access by index	Fast, $O(1)$	Slow, $O(n)$
Insert / remove in the middle	Slower, $O(n)$	Faster

- Project link:** BitInsight uses `ArrayList` (e.g. `new ArrayList<>()`) for coin lists in `Watchlist` because items are mostly read and displayed in order.

Q4. How does `HashMap` work internally?

- **Short answer:** `HashMap` stores key-value pairs in an array of buckets. On `.put(key, value)`, Java computes the key's `hashCode()` to choose the bucket index. If several keys land in the same bucket (a **collision**), they are stored as a linked list, or as a balanced **Red-Black Tree** in Java 8+ if the bucket grows beyond 8 items. On `.get(key)`, Java finds the bucket and uses `.equals()` to return the matching value.

Q5. What is the difference between checked and unchecked exceptions?

Type	Checked at	Examples	Handling
Checked	Compile time	<code>IOException</code> , <code>SQLException</code>	Must use <code>try-catch</code> or declare <code>throws</code>
Unchecked (Runtime)	Run time	<code>NullPointerException</code> , <code>ArithmeticException</code>	Inherit from <code>RuntimeException</code> . Handling is not mandatory.

- **Project link:** Spring's `@Transactional` rolls back automatically on **unchecked** exceptions by default.

Q6. What are Java 21 Virtual Threads?

- **Short answer:** Virtual Threads are very lightweight threads managed by the JVM instead of the operating system. Traditional OS threads take about 1 MB of memory and are limited to a few thousand per machine. Virtual Threads use only a few hundred bytes, so one Spring Boot application can run millions of concurrent tasks with little memory.

Q7. What is the difference between `Comparable` and `Comparator` ?

	<code>Comparable</code>	<code>Comparator</code>
Purpose	One natural sorting order for a class	Multiple custom sorting strategies
Where defined	Inside the class	Separate class or lambda
Method	<code>compareTo()</code>	<code>compare()</code>

Q8. What is the Java Stream API and why is it used?

- **Short answer:** The Stream API (Java 8) processes collections declaratively and functionally, using operations such as `filter()`, `map()`, `sorted()`, and `collect()`. It makes code cleaner, removes boilerplate `for` loops, and supports easy parallel processing with `.parallelStream()`.

34 Spring Boot Questions

Q1. What is Spring Boot and how does it differ from traditional Spring?

- **Short answer:** Spring Boot extends Spring to remove boilerplate configuration. In traditional Spring you configure XML files, `DispatcherServlet`, view resolvers, and an external Tomcat by hand. Spring Boot provides **Auto-Configuration**, **Starter Dependencies**, and an **Embedded Tomcat Server**, so you can run a production-ready app with a single Java command.

Q2. What are Dependency Injection (DI) and Inversion of Control (IoC)?

- **Inversion of Control (IoC):** Instead of creating objects with `new`, the Spring container creates, manages, and destroys them for you.
- **Dependency Injection (DI):** The mechanism Spring uses to pass required dependencies into a class at runtime (e.g. injecting `UserRepository` into `UserServiceImpl`).

Q3. What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?

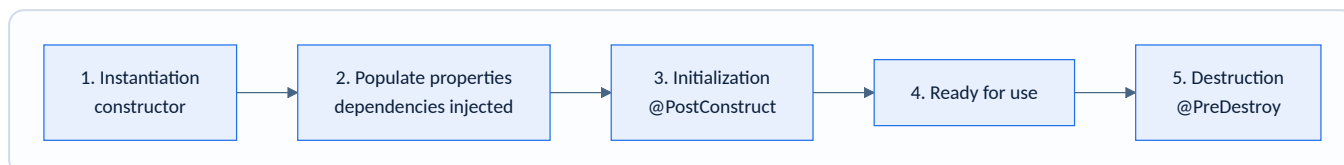
All four register a class as a Spring-managed bean, but each has a specific role:

Annotation	Role
<code>@Component</code>	General-purpose bean annotation for any Spring-managed class
<code>@Service</code>	Business logic classes in the service layer
<code>@Repository</code>	Data access classes. Also enables automatic translation of SQL exceptions into Spring's <code>DataAccessException</code> hierarchy.
<code>@Controller</code> / <code>@RestController</code>	Classes that handle HTTP requests and REST APIs

Q4. Why is constructor injection preferred over `@Autowired` field injection?

1. **Immutability:** dependencies can be `final`, so they can't change after creation.
2. **Easy unit testing:** you can pass mock objects through the constructor without reflection or a Spring context.
3. **Circular dependencies:** Spring detects them immediately at startup instead of failing at runtime.

Q5. What is the Spring bean lifecycle?



Q6. What is the difference between `@RequestParam` and `@PathVariable`?

Annotation	Reads from	Example
<code>@PathVariable</code>	The URI path	<code>/coins/{coinId}</code> → <code>/coins/bitcoin</code> (<code>coinId = "bitcoin"</code>)
<code>@RequestParam</code>	Query parameters after <code>?</code>	<code>/coins?page=2</code> (<code>page = 2</code>)

Q7. How does `@RequestBody` work with JSON deserialization?

- **Short answer:** `@RequestBody` tells Spring to take the HTTP request body (a JSON string) and convert it into a Java object using the Jackson `ObjectMapper`.

Q8. What are Spring Boot Starters?

- **Short answer:** Starters are dependency descriptors you add to `pom.xml`. Instead of finding and matching ten compatible libraries, adding `spring-boot-starter-web` pulls in Tomcat, Jackson, Spring MVC, and validation.

libraries with compatible versions.

35 Spring Security Questions

Q1. How does Spring Security work at a high level?

- **Short answer:** Spring Security sits between incoming HTTP requests and your REST controllers as a chain of servlet filters called the `SecurityFilterChain`. Every request passes through the filters in order. They check whether the request is public or protected, extract authentication tokens, verify identity, and enforce roles before the request reaches your controller.

Q2. What is `SecurityContextHolder` and where is authentication stored?

- **Short answer:** It's the central place where Spring Security keeps the details of the currently authenticated user. It uses a `ThreadLocal` strategy, so each request thread has its own isolated security context holding the user's `Authentication` object.

Q3. What is the difference between `Authentication` and `Principal` ?

Term	Meaning
Principal	The identity of the user (e.g. email, username, or <code>UserDetails</code> object)
Authentication	The full token object containing the Principal, the credentials (password/token), and the assigned <code>GrantedAuthority</code> values (roles/permissions)

Q4. What is the role of `CustomUserDetailsService` in this project?

- **Short answer:** It implements Spring Security's `UserDetailsService` interface and defines `loadUserByUsername(String email)`. Spring calls it during login to fetch the user from MySQL and verify the submitted credentials.

Q5. What is CORS and why does the browser block requests?

- **Short answer:** **Cross-Origin Resource Sharing (CORS)** is a browser security policy. It stops a page on one origin (e.g. React on `localhost:5173`) from calling an API on a different origin (e.g. Spring Boot on `localhost:5454`) unless the backend returns headers permitting that origin.

Q6. Why is CSRF disabled in a stateless JWT architecture?

- **Short answer:** CSRF attacks trick the browser into automatically sending stored **session cookies** with unauthorized requests. BitInsight is stateless and authenticates through an `Authorization: Bearer <token>` header that browsers **never attach automatically**, so CSRF protection is unnecessary.

Q7. What is a CORS preflight (`OPTIONS`) request?

- **Short answer:** Before a non-simple request (one with custom headers such as `Authorization`, or verbs such as `PUT` or `DELETE`), the browser automatically sends a lightweight `OPTIONS` request. The server must reply with the permitted origins, methods, and headers before the browser sends the real request.

36 JWT Questions

Q1. What is a JWT and why is it used?

- **Short answer:** A **JSON Web Token** is a compact, URL-safe token for securely transmitting claims between client and server. It enables **stateless authentication**: after login the server generates a token and keeps no session. The client sends the token on later requests, and the server validates it cryptographically without querying the database for session state.

Q2. What are the three parts of a JWT?

A JWT has three parts separated by dots (.):

Part	Contents
Header	Token type (<code>JWT</code>) and signing algorithm (e.g. <code>HS256</code>)
Payload	Claims (e.g. user email, roles, expiration timestamp)
Signature	The Header and Payload hashed together with a secret key held only by the server

Q3. Can anyone decode and read the contents of a JWT?

- **Short answer: Yes.** The header and payload are only **Base64Url-encoded, not encrypted**. Anyone who intercepts a token can read the email or roles. So **never** put secrets such as plain passwords or credit card numbers in a JWT payload.

⚠ Common Mistake

Saying "JWT encrypts user data." A JWT is **signed**, not encrypted.

Q4. If anyone can read a JWT, how does the server know it hasn't been tampered with?

- **Short answer:** Through the **Signature**. The server re-computes the signature from the received Header, Payload, and its secret key. If an attacker changed the email or role, the re-computed signature won't match the token's signature, and the server rejects the token.

Q5. What is the difference between an access token and a refresh token?

Token	Lifetime	Purpose
Access token	Short-lived (e.g. 15 minutes to 24 hours)	Accesses protected APIs directly
Refresh token	Long-lived, stored securely	Used only to request a new access token when the old one expires, so the user doesn't re-enter the password

✓ Remember

BitInsight uses a **single 24-hour access token** and has **no refresh tokens** yet.

Q6. Where should a JWT be stored on the client?

Storage	Pros and cons
<code>localStorage</code>	Easy to implement but vulnerable to XSS . <i>This is what the current React client uses.</i>
HttpOnly cookie (best practice)	Stored with the <code>HttpOnly</code> and <code>Secure</code> flags, so JavaScript can't read it, which mitigates XSS token theft.

Q7. How can you revoke or log out a user before the JWT expires?

- **Short answer:** Because JWT is stateless, the server can't naturally destroy a token. Common solutions:
 1. Store revoked token IDs in a fast in-memory **Redis blacklist** with a TTL equal to the token's remaining lifespan.
 2. Keep a `tokenVersion` or `jwtIssuedAfter` timestamp on the user's database record and invalidate any token issued earlier.

37 Database and SQL Questions

Q1. What is the difference between a primary key and a foreign key?

Key	Definition	Example
Primary key	Unique identifier of a row. Cannot be <code>NULL</code> .	<code>user.id</code>
Foreign key	A column that references another table's primary key, creating a relationship and maintaining referential integrity.	<code>wallet.user_id</code> → <code>user.id</code>

Q2. What is the difference between `DELETE`, `TRUNCATE`, and `DROP`?

Command	Type	Behaviour
<code>DELETE</code>	DML	Deletes specific rows matching a <code>WHERE</code> clause. Can be rolled back and fires triggers.
<code>TRUNCATE</code>	DDL	Removes all rows at once. Faster than <code>DELETE</code> because it deallocates data pages directly. Cannot be rolled back easily.
<code>DROP</code>	DDL	Deletes the entire table structure and its data from the schema.

Q3. What is a database index and how does it speed up queries?

- **Short answer:** An index is a data structure (typically a **B-Tree**) that works like the index at the back of a textbook. Instead of a full table scan ($O(n)$), the database searches the index in $O(\log n)$ time to find the exact disk page holding the row.

Q4. What is the difference between `INNER JOIN` and `LEFT JOIN`?

Join	Result
<code>INNER JOIN</code>	Only rows with a match in both tables

Join	Result
LEFT JOIN	All rows from the left table, plus matching rows from the right. Where there is no match, right-table columns are NULL .

Q5. What are the ACID properties?

Property	Meaning
Atomicity	All operations in a transaction succeed together, or all fail and roll back (all-or-nothing).
Consistency	The database moves from one valid state to another, respecting all constraints.
Isolation	Concurrent transactions don't see each other's uncommitted changes.
Durability	Once committed, changes are permanently stored and survive system crashes.

Q6. What is database normalization and why is it used?

- **Short answer:** Normalization organizes tables to reduce redundant data and prevent update, insertion, and deletion anomalies. In BitInsight, user credentials, wallets, orders, and bank details are split into separate relational tables connected by foreign keys.

Q7. Why should financial balances use **BigDecimal** instead of **double** or **float**?

- **Short answer:** **double** and **float** are binary floating-point types made for scientific calculations. They can't represent exact decimal fractions (e.g. $0.1 + 0.2 = 0.30000000000000004$). In a trading app, rounding errors could cause lost money or ledger discrepancies. **BigDecimal** gives exact precision with configurable rounding modes.

38 JPA and Hibernate Questions

Q1. What is the difference between JPA and Hibernate?

- **JPA (Jakarta Persistence API)** is a **specification**: a set of interfaces and rules for mapping Java objects to database tables.
- **Hibernate** is the **implementation engine** that implements those interfaces and runs the underlying SQL.

Q2. What is an Entity in JPA?

- **Short answer:** A lightweight Java class annotated with **@Entity** that represents a database table. Each instance corresponds to one row.

Q3. What is the N+1 query problem in Hibernate and how do you fix it?

- **Short answer:** Fetching 1 parent triggers 1 SQL query, then fetching its children triggers **N** additional queries (e.g. 100 orders cause 101 queries). Fix it with **JOIN FETCH** in JPQL or with **@EntityGraph**, which load parent and children in a single **JOIN**.

Q4. What is the difference between `findById()` and `getReferenceById()` ?

Method	Behaviour
<code>findById()</code>	Runs the SQL query immediately and returns the real entity wrapped in an <code>Optional</code> .
<code>getReferenceById()</code>	Returns a lazy Hibernate proxy without an immediate query. The real query runs only when a field is accessed.

Q5. What is `CascadeType` in JPA?

- **Short answer:** Cascade defines which operations on a parent entity automatically propagate to its children. For example, `cascade = CascadeType.ALL` means saving, updating, or deleting an `Order` also saves, updates, or deletes its `OrderItem` .

Q6. What is the difference between `@OneToMany` and `@ManyToMany` ?

Annotation	Meaning	Example
<code>@OneToMany</code>	One parent has many children	One <code>User</code> has many <code>Order</code> records
<code>@ManyToMany</code>	Many records relate to many records. Needs a <code>join table</code> .	Many <code>Watchlist</code> entities contain many <code>Coin</code> records (join table <code>watchlist_coins</code>)

Q7. What is dirty checking in Hibernate?

- **Short answer:** When you load an entity inside a `@Transactional` method, Hibernate keeps a snapshot of it. When the transaction finishes, Hibernate compares the current state with the snapshot. If any field changed, it automatically runs a SQL `UPDATE` , even without calling `repository.save()` .

Q8. What is the difference between Hibernate first-level and second-level cache?

Cache	Scope	Details
First-level	Current Hibernate <code>Session</code> (one transaction)	Enabled by default. Caches entities within the same request.
Second-level	Application-wide, shared across sessions	Optional. Often implemented with Redis or Ehcache.

39 React and Frontend Questions

Q1. What is React and why do we use it?

- **Short answer:** React is a component-based JavaScript library from Meta for building user interfaces. Its declarative approach and reusable components make it easy to build dynamic single-page applications with fast updates through the **Virtual DOM**.

Q2. What is JSX?

- **Short answer:** JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup inside JavaScript. At build time, tools like Babel/Vite compile it into standard `React.createElement()` calls.

Q3. What is the difference between `useState` and `useEffect` ?

Hook	Purpose
<code>useState</code>	Lets functional components store and update local state, triggering a re-render on change
<code>useEffect</code>	Performs side effects (fetching data, timers, DOM manipulation) after the component renders

Q4. What is the difference between props and state?

	Props	State
Direction	Passed from parent to child	Managed inside the component
Mutability	Read-only for the receiving child	Mutable over time through user interaction

Q5. What is Redux and why did you use it in BitInsight?

- **Short answer:** Redux is a predictable global state-management library. BitInsight keeps one central store for global state, specifically **authentication** (`auth`) and **market coin lists** (`coin`). This prevents *prop drilling*, where data would otherwise be passed manually through dozens of nested components.

Q6. What is Redux Thunk and why is it needed?

- **Short answer:** By default, Redux reducers and action creators must be pure and synchronous. **Redux Thunk** is middleware that lets an action creator return a **function** instead of an action object. That enables asynchronous work (such as `api.get("/api/wallet")` with Axios) and dispatching actions when the response arrives.

Q7. Why use Axios instead of the native `fetch()` ?

1. Axios automatically serializes and deserializes JSON (no `.json()` call).
2. Axios supports centralized instances with base URLs and request/response **interceptors**, ideal for attaching Bearer tokens automatically.
3. Axios throws errors automatically on HTTP 4xx and 5xx, whereas `fetch()` rejects only on network failures.

Q8. What are React keys and why are they required in lists?

- **Short answer:** When rendering a list (e.g. mapping over coins in a table), each element needs a unique `key` prop. Keys help the Virtual DOM identify which items changed, were added, or were removed, avoiding unnecessary re-rendering of the whole list.

40 Docker and Deployment Questions

✓ Remember

Docker, Nginx, and CI/CD are **not implemented** in the repository. Answer these as *concepts you understand and a plan for BitInsight*, as in [Section 21](#) to [Section 23](#).

Q1. What is Docker and what problem does it solve?

- **Short answer:** Docker is a containerization platform that packages an application with its dependencies, runtime, and configuration into a single portable **container**. It solves the classic problem: *"It works on my machine, but not on the production server."*

Q2. What is the difference between a Docker image and a Docker container?

Term	Definition	Java analogy
Image	A read-only template with the code, libraries, and instructions to run an application	A class
Container	A running, isolated instance of an image	An instantiated object

Q3. What is a Dockerfile ?

- **Short answer:** A text file with step-by-step commands Docker runs to build an image. Common commands: `FROM` (base image), `COPY` (copy files), `RUN` (execute build commands), `EXPOSE` (declare ports), and `ENTRYPOINT` (command to run at container start).

Q4. What is Docker Compose?

- **Short answer:** A tool for defining and running multi-container applications in a single YAML file (`docker-compose.yml`). With `docker compose up` you can start the Spring Boot backend, the MySQL database, and the React frontend together on a shared virtual network.

Q5. What is an Nginx reverse proxy?

- **Short answer:** A web server between external clients and your backend services. It receives client traffic, terminates SSL/HTTPS, serves static frontend files, and forwards API calls (e.g. `/api/*`) to internal Spring Boot ports, so backend servers are not exposed directly to the internet.

Q6. What is the difference between CI and CD?

Term	Meaning
Continuous Integration (CI)	Automatically builds the code and runs unit/integration tests every time a developer pushes to GitHub.
Continuous Deployment (CD)	Automatically packages the tested application into Docker containers and deploys to staging or production without manual intervention.

41 Mock Interview Simulation

A sequential **30-question track**. Read each question aloud, answer without notes, then check the target.

Interview Tip

Record yourself answering Q1 to Q10 and listen back. If you hear long pauses or filler words, rewrite that answer in shorter sentences using the *Short answer* style from Part VI.

#	Interviewer question	What the interviewer is looking for
1	"What does this application do?"	A high-level elevator pitch. ⚠️ <i>Common mistake:</i> diving straight into database tables instead of explaining the business purpose.
2	"Walk me through the technologies used."	Full-stack awareness.
3	"Why did you use Spring Boot 3.4 instead of older versions?"	Understanding framework evolution and the Java 21 LTS baseline.
4	"Explain the login flow from React to the database."	Tracing the end-to-end request lifecycle.
5	"Where is the JWT generated and validated?"	<code>JwtProvider</code> and <code>JwtTokenValidator</code> .
6	"How does Two-Factor Authentication work in your system?"	OTP generation, email dispatch, and the temporary session ID.
7	"How do you protect your endpoints?"	The <code>SecurityFilterChain</code> in <code>AppConfig</code> .
8	"What happens when someone calls <code>/api/orders/pay ?</code> "	Transactional order placement.
9	"How do you ensure data integrity during a crypto purchase?"	<code>@Transactional</code> boundaries.
10	"How do you fetch real-time crypto prices?"	The CoinGecko API integration.
11	"CoinGecko has rate limits. How did your backend handle this?"	The Caffeine caching strategy and the <code>@Scheduled</code> background worker.
12	"Why did you choose Caffeine over Redis?"	Trade-offs between in-process and distributed caches.
13	"How does fiat deposit work?"	Razorpay and Stripe SDKs and callback verification.
14	"How does a user withdraw funds to their bank?"	The <code>Withdrawal</code> lifecycle from <code>PENDING</code> to <code>SUCCESS</code> .
15	"How are roles modeled in your database?"	The <code>USER_ROLE</code> enum and the user table structure.
16	"What happens if a user tries to access admin withdrawal approvals?"	An honest evaluation of the current authorization rules and the recommended improvement.
17	"Explain the relationship between User, Wallet, and Asset."	1:1 and 1:N JPA mappings.
18	"What is the difference between <code>@OneToOne</code> and <code>@ManyToOne</code> ?"	Relational modeling fundamentals.
19	"How do you handle exceptions across your REST APIs?"	<code>@ControllerAdvice</code> and <code>GlobalExceptionHandler</code> .
20	"What HTTP status codes does your API return?"	REST conventions: 200, 201, 202, 400, 429, 500.
21	"How does your React frontend manage global state?"	Redux, Redux Thunk, actions, and reducers.
22	"How does Axios pass the authentication token?"	The <code>authHeaders()</code> helper and <code>Authorization: Bearer <token></code> .

#	Interviewer question	What the interviewer is looking for
2 3	"How do you render the price trend graphs?"	ApexCharts and coordinate mapping.
2 4	"What is CORS and how did you configure it?"	Browser origin policy and <code>CorsConfigurationSource</code> .
2 5	"How do you store user passwords currently?"	An honest admission of plain-text storage and a clear plan for <code>BCryptPasswordEncoder</code> .
2 6	"How would you deploy this project to production?"	Docker, Nginx, and cloud virtual machines (as a plan).
2 7	"What was the most challenging bug you fixed?"	A problem-solving story (e.g. CoinGecko 429 rate limits or transactional rollback).
2 8	"What would you improve if given another sprint?"	High-value improvements: password hashing, RBAC, WebSockets.
2 9	"How do you test your application?"	Unit testing with JUnit 5 and Mockito, and Postman manual testing. ⚠️ Automated testing is listed under Future Improvements , so describe it honestly.
3 0	"Why should we hire you based on this project?"	A confident closing statement that connects your hands-on code experience with production readiness.

PART VII

Revision Toolkit

42 Technical Glossary

#	Term	Meaning
1	API (Application Programming Interface)	A structured protocol that lets different software systems communicate.
2	REST (Representational State Transfer)	An architectural style for networked applications using standard HTTP verbs.
3	Controller	A Spring component that receives HTTP requests and returns responses.
4	Service	A Spring layer that holds business logic and transaction boundaries.
5	Repository	A data-access abstraction providing CRUD operations on database entities.
6	Entity	A Java class mapped to a relational database table with JPA annotations.
7	DTO (Data Transfer Object)	An object that carries data between processes, preventing direct exposure of internal entities.
8	JWT (JSON Web Token)	A compact, URL-safe way to represent claims securely between two parties.
9	CORS (Cross-Origin Resource Sharing)	A browser security mechanism that restricts cross-origin HTTP requests.
10	CSRF (Cross-Site Request Forgery)	An attack that forces authenticated users to perform unwanted actions on a web application.
11	ACID	Atomicity, Consistency, Isolation, Durability: the guarantees of relational database transactions.
12	ORM (Object-Relational Mapping)	A technique for converting data between relational databases and object-oriented languages.
13	Caffeine Cache	A high-performance, near-optimal Java caching library that runs directly in JVM memory.
14	TTL (Time-To-Live)	How long cached data lives before it expires and is invalidated.
15	Stateless session	An approach where the server keeps no client state between requests.
16	HMAC-SHA	A message authentication code that combines a cryptographic hash function with a secret key.
17	Spring AOP (Aspect-Oriented Programming)	Modularizes cross-cutting concerns such as security and transactions.
18	HikariCP	A high-performance JDBC connection-pool library, used by default in Spring Boot.
19	Redux Thunk	Middleware that lets action creators return a function instead of an action, for asynchronous logic.
20	ApexCharts	An interactive charting library for modern web applications.

43 Things Not to Say in an Interview

⚠ Common Mistake

Review these pitfalls carefully. One over-claim can undermine your credibility for the entire interview.

#	❌ Do not say	✅ Reality (say this instead)
1	"I built this with microservices and Redis."	The project is a monolithic Spring Boot application with an in-memory Caffeine cache. Be proud of the architecture you actually built.
2	"I used BCrypt to securely encrypt all passwords."	The code currently stores passwords in plain text . State honestly that password hashing with BCrypt is the priority for the next release.
3	"I deployed this to a Kubernetes cluster on AWS."	No Kubernetes manifests exist in the repository. Explain how you ran it locally and how you would containerize it with Docker.
4	"Spring Data JPA is my database."	MySQL is the database. Spring Data JPA is an abstraction over Hibernate, which is the ORM provider.
5	"JWT encrypts user data so nobody can read it."	Standard JWT payloads are Base64Url-encoded and readable by anyone. They are digitally signed, not encrypted .
6	"I used Firebase for push notifications."	Notifications are sent by email through JavaMailSender . Firebase is not integrated.

44 My Complete Project Story

🎯 Interview Tip

Use this when the interviewer says "Walk me through how you built it." It follows a natural arc: **goal → design → hardest problem → integrations → frontend → what you learned**.

When I started **BitInsight**, my goal was to build a clean, accessible crypto trading platform without the clutter of traditional exchanges, while still keeping the money handling reliable.

I started by planning the data model: users, wallets, crypto holdings, orders, and payment records. I chose **Java 21** and **Spring Boot 3.4** for the backend because of its enterprise-grade ecosystem, and **MySQL** for strict ACID transactional consistency.

As I built the core trading logic, my biggest challenge was **transactional integrity**. When a user buys Bitcoin, we have to deduct wallet funds, record the order, and add to their asset holdings together. I put that logic inside Spring's **@Transactional** boundary, so any failure rolls back the whole operation.

When I integrated the **CoinGecko API** for live prices, I quickly ran into **HTTP 429 rate limits**. Instead of adding extra infrastructure like Redis, I added an in-memory **Caffeine cache** with a 5-minute TTL, plus a background **@Scheduled** task that refreshes the cache every 10 minutes.

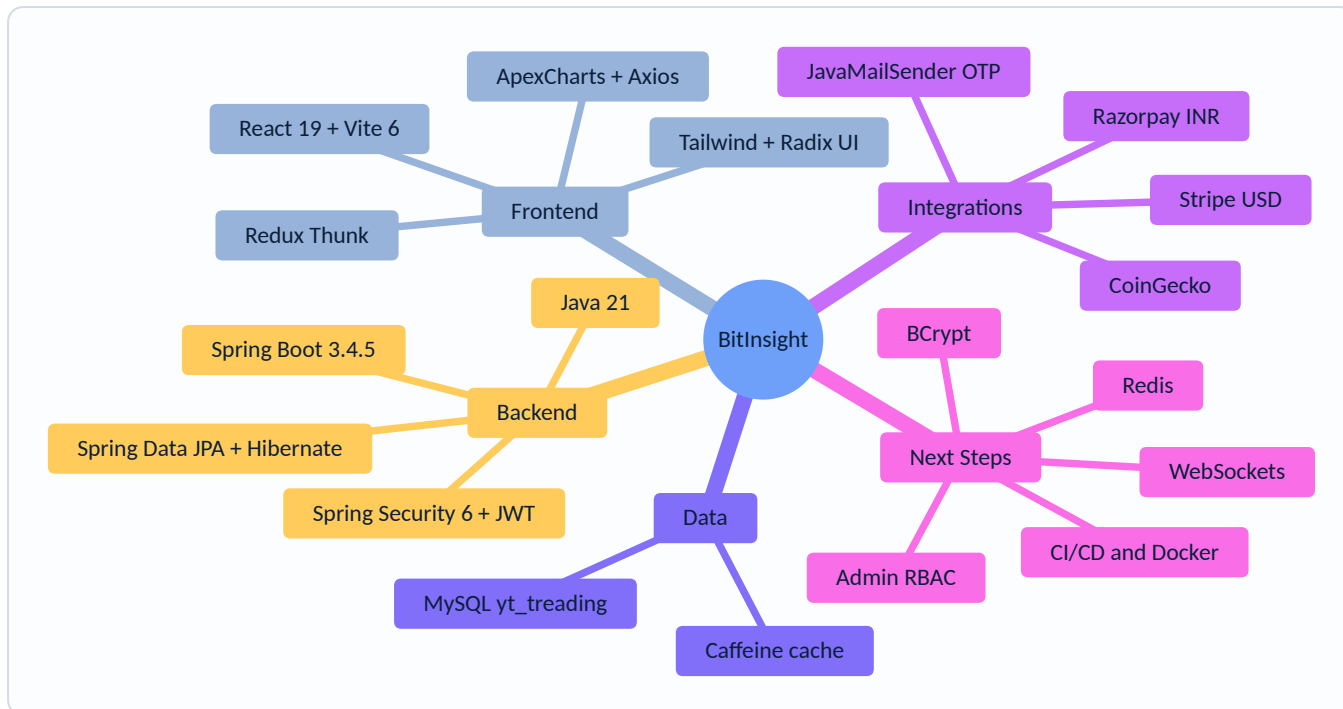
For payments, I integrated **Razorpay** and **Stripe** so users can deposit money easily, and I added **email-based Two-Factor Authentication** to secure logins.

On the frontend, I used **React 19**, **Redux**, and **Tailwind CSS**, with interactive **ApexCharts** to show market trends.

This project gave me hands-on experience with transactional business logic, REST API design, caching strategies, and secure full-stack development.

45 One-Page Quick Revision

Project Mind Map



Cheat Sheet

Topic	Key facts
Project	BitInsight (Crypto Trading & Wallet Platform)
Architecture	Monolithic REST API backend + React SPA frontend
Backend stack	Java 21, Spring Boot 3.4.5, Spring Security 6, Spring Data JPA, Hibernate
Frontend stack	React 19, Vite 6, Redux Thunk, Tailwind CSS, Radix UI, ApexCharts
Database	MySQL (<code>yt_treading</code> , port 3306)
Caching	Caffeine (<code>coinList</code> , <code>marketChart</code>), 5-minute TTL, plus <code>@Scheduled</code> refresh every 10 minutes
Authentication	Stateless JWT (HMAC-SHA, 24-hour expiry) + email OTP 2FA
Third-party APIs	CoinGecko API v3 (market data), Razorpay (INR), Stripe (USD), JavaMailSender
Ports	Backend 5454 · Frontend 5173 / 5174 · Database 3306

Core Entities

Entity	Key fields
<code>User</code>	id, email, password, role, twoFactorAuth
<code>Wallet</code>	id, user_id, balance (<code>BigDecimal</code>)
<code>Asset</code>	id, user_id, coin_id, quantity, buyPrice
<code>Coin</code>	id, name, symbol, currentPrice, marketCap, marketCapRank

Entity	Key fields
Order	id, user_id, orderType (BUY/SELL), price, status, timestamp
OrderItem	id, order_id, coin_id, quantity, buyPrice, sellPrice
Watchlist	id, user_id, coins (ManyToMany)
PaymentOrder	id, user_id, amount, paymentMethod, status
Withdrawal	id, user_id, amount, status, date
PaymentDetails	id, user_id, accountNumber, accountHolderName, ifsc, bankName

Three Key Highlights

1. **ACID order execution:** `@Transactional` buy/sell logic prevents balance inconsistencies.
2. **Resilience caching:** the Caffeine cache avoids CoinGecko 429 rate-limit errors.
3. **Dual payment gateway:** domestic (Razorpay) and global (Stripe) top-ups.

Honest Gaps to Mention Yourself

Gap	Fix
Plain-text passwords	<code>BCryptPasswordEncoder</code>
Admin routes only require authentication	<code>.requestMatchers("/api/admin/**").hasRole("ADMIN")</code>
In-process cache	Redis when scaling to multiple instances
REST polling for prices	WebSocket (STOMP) feeds
No automated tests, Docker, or CI/CD	JUnit 5 + Mockito, Dockerfiles + Compose, GitHub Actions

46 Top 20 Interview Questions

Interview Priority

Memorize these twenty answers. Together they cover most of what a project-based fresher interview will ask.

#	Question	Answer
1	What is the project?	A full-stack crypto trading platform with a Spring Boot REST backend, a MySQL database, and a React frontend.
2	How does authentication work?	Stateless JWTs signed with HMAC-SHA, valid for 24 hours, with optional email OTP 2FA.
3	What is the database?	MySQL, accessed through Spring Data JPA and Hibernate ORM.
4	How do you prevent CoinGecko rate limiting?	An in-memory Caffeine cache (5-minute TTL) with a background <code>@Scheduled</code> refresher every 10 minutes.
5	How does buying crypto work?	In a <code>@Transactional</code> method, the system verifies the wallet balance, deducts funds, records an Order and OrderItem, and creates or updates an Asset holding.
6	How does selling crypto work?	It verifies asset quantity, calculates proceeds, decreases or deletes the Asset holding, records the Order, and credits the wallet.

#	Question	Answer
7	What happens if an order fails midway?	The <code>@Transactional</code> boundary triggers a rollback, reverting any wallet deduction or asset update.
8	Why Caffeine instead of Redis?	Caffeine runs in JVM memory with zero network latency and no external infrastructure, which suits a single-instance monolith.
9	How do fiat deposits work?	The user starts a payment through Razorpay or Stripe. After the gateway completes, a callback confirms the payment and credits the wallet.
10	How does wallet-to-wallet transfer work?	The sender gives a recipient wallet ID and amount. The sender's balance is validated, deducted, and credited to the recipient.
11	How do withdrawals work?	The user saves bank details and submits a request. Funds are deducted and the request waits for admin approval.
12	What roles exist?	<code>ROLE_CUSTOMER</code> and <code>ROLE_ADMIN</code> .
13	What is the purpose of <code>JwtTokenValidator</code> ?	A <code>OncePerRequestFilter</code> that intercepts requests, validates the Bearer token, and populates <code>SecurityContextHolder</code> .
14	How is CORS handled?	Configured in <code>AppConfig</code> to allow <code>http://localhost:5173</code> , <code>5174</code> , and <code>3030</code> with credentials and an exposed <code>Authorization</code> header.
15	How is error handling structured?	Centrally through <code>@ControllerAdvice</code> in <code>GlobalExceptionHandler</code> , which maps exceptions to clean <code>ApiResponse</code> objects.
16	Why use <code>BigDecimal</code> for currency?	<code>float</code> and <code>double</code> introduce rounding errors because of binary representation. <code>BigDecimal</code> guarantees exact decimal precision.
17	How does the frontend manage state?	Redux manages global authentication and coin lists. Local React state handles page-specific forms and dialogs.
18	How are market charts rendered?	ApexCharts renders historical coordinates fetched from CoinGecko through the backend.
19	What key security improvement is needed?	Implement <code>BCryptPasswordEncoder</code> to hash passwords instead of storing them in plain text.
20	What key architectural improvement is needed?	Move to Redis distributed caching and WebSocket real-time price feeds when scaling horizontally.

47 Final Interview Checklist

Before the interview, make sure you can do each of these **without looking at notes**.

Pitch and Story

- ☐ Deliver the **1-minute project pitch** naturally, in your own words.
- ☐ Tell the **project story**: goal, design, hardest problem, integrations, lessons.
- ☐ State your **specific contribution** in two sentences.

Architecture and Flows

- ☐ **Draw the system architecture** on a whiteboard (React → filter → controller → service → repository → MySQL, plus external APIs).
- ☐ Explain the **login and JWT lifecycle** from the HTTP request to `SecurityContextHolder`.
- ☐ Walk through the **buy-asset order execution** and explain why `@Transactional` is critical.

- ☐ Explain how **Razorpay and Stripe** integrations credit the wallet balance.
- ☐ Explain the **withdrawal lifecycle** from `PENDING` to admin approval.

Technology Choices

- ☐ Explain why **Caffeine** was chosen and how `@Scheduled` cache warming works.
- ☐ Detail the differences between **Spring Boot, Spring Security, JPA, and Hibernate**.
- ☐ Clearly explain the **entity relationships** (`User` , `Wallet` , `Asset` , `Order` , `Coin`).
- ☐ Explain **CORS, CSRF, and why CSRF is disabled** for stateless JWT.

Honesty and Improvements

- ☐ Describe the difference between your **current implementation** and **future improvements** (BCrypt password hashing, admin RBAC, Redis, WebSockets).
- ☐ Say plainly that **Docker, CI/CD, and Redis are not implemented**, and describe your plan for each.
- ☐ Know the **"Do not say" list** in [Section 43](#).

Final Readiness Check

Check	Done
I can answer all 🔥 High Priority questions in Section 31	☐
I can answer the Top 20 in Section 46 from memory	☐
I completed the 30-question mock in Section 41 out loud	☐
I know exact names: <code>modal</code> package, <code>CustomUserDetailsService</code> , <code>JwtTokenValidator</code> , <code>CacheRefreshScheduler</code>	☐
I can say which features are ✅ Implemented, 🟡 Partial, and 🟠 Future	☐

✅ Remember

Interviewers hire people they can trust. A candidate who clearly says *"here is what I built, here is what is missing, and here is how I would fix it"* sounds more senior than one who claims everything works perfectly.

Good luck with your interview! 🚀

BitInsight · Java 21 · Spring Boot 3.4.5 · React 19